Where the Capital Bound Moves:

Settlement Objects, Proof Economics, and Agent Coordination on Bitcoin Payment Networks

Patrick B. Dugan*

October 2026
Draft for comment, revised October 16, 2026

Abstract

A population of autonomous agents that pays at high frequency from low balances, and is always online, meets a cost on the Lightning Network that no routing policy reduces: inbound capacity toward each agent has to be provisioned before its demand is known, and the provisioning is a forecast made per channel, with no pooling across agents. We state this as a prefix cut bound on a directed graph with hidden directional balances, show that it is invariant to routing policy, and report a simulator experiment in which routing intelligence is worth between a third of a point and one point of payment success and capital placement is worth six. We then ask where the bound goes under a server-mediated settlement object of the Ark type. It pools: the per-edge directional constraint becomes an aggregate constraint on the server's uncommitted capital, and the directional forecast moves to the server's own channels rather than vanishing. A liquidity-duration ratio states when pooling pays. On a reference model the answer turns on whether value a server fronts at a Lightning spend is recoverable at the next round or only at expiry, and the implementer's server takes the second: the output it receives at a spend is swept on chain only after its round expires, and receipts are issued from a separate inventory of server outputs funded on chain with a lifetime of days. The virtual output is favored by a factor between one and two and a half under the first reading, on populations whose per-agent imbalances are uncorrelated, with the factor set by how conservatively the channel is provisioned, and not favored under the second, which is the one deployed. A third object is not yet available. The soft fork proposed as BIP 448 would make multiparty channels practical, and a channel shared by k agents and a hub turns the edge into a hyperedge: the cut bound survives unchanged, and the forecast pools inside each group with no server and no forfeit lock, at the price of every member's signature and of exposure to every other member's departure. On the reference model a group of twenty captures most of what a server pools, and because agents that start from nothing hold balances that do not net, what there is to capture is a constant near $\pi/2$ and not $\sqrt{N}$. A fourth object, a covenant tree proposed in October 2026 under the name LayerTree, has no forfeit lock and no expiry on its output, places its operator's stock per member rather than pooled and pays for a shorter forecast horizon in bytes on chain, and brings a throughput bound on its single output. For the reward layer such a population would need, we give a claim-once registry keyed by an object's spend authority that bounds the weight any settled object can contribute, show that a nullifier alone leaves spend-and-reclaim open and that an epoch rule closes it, and set out the cost of verifying such claims on Bitcoin, where the verifier is a trusted party or an optimistic game whose fixed cost is paid per

*Spiral program. Model, fixtures, and preregistrations: https://github.com/patrickdugan/Spiral. Correspondence: patrick.buchanan.dugan@gmail.com.

assertion. Finally we ask what a population of agents under common control could do with these objects. An observer model fixes who sees what under each; a coalition's gain from coordinating is bounded by its authenticated capital placed under stationary demand, at the scale the placement experiment measured, whatever the capacity of its covert channel, so the steganographic question is second-order to the capital question. A swarm that runs its own server collapses its observable surface to round transactions and ramps; enforceable escrows give it a commitment device cartels have historically lacked; and agents instantiated from the same weights coordinate without a message, in a way that payment traces cannot distinguish from independent use of one model. We preregister the experiments that test each claim. All results are from a reference model with a simulated proof system.

1 Introduction

Consider a population of software agents that transact with one another and with services over Bitcoin. Each agent holds a small balance, pays often, receives often, and is online continuously; none has a bank, a reputation, or a legal identity, and the population changes membership faster than any bilateral relationship can be amortized. The Lightning Network [42] is the obvious rail, and the question this paper starts from is what it costs such a population to use it.

The answer the routing literature suggests is that the cost is in finding paths through a graph whose edge capacities are hidden [41, 56]. That is a cost, and an agent that probes, learns, and retries pays less of it than a wallet that does not. But it is not the cost that binds. What binds is that a Lightning channel is bilateral, pre-funded, and partitioned by direction: each unit of capacity belongs to one edge, is committed before any payment is known, and sits on one side of the channel or the other. An agent that is going to be paid needs inbound capacity on some channel, and that capacity is outbound capacity for whoever is on the other end, who had to put it there by a forecast. No routing policy relaxes this, because it is a conservation identity over the cut around the agent and not a property of path selection.

Section 3 states that identity as a lemma on a directed graph with hidden directional balances, shows that it holds for every routing policy including one with full knowledge of balances, and reports what a preregistered simulator campaign on sampled public topology found: bounded retries are worth about five points of success over one-shot routing, learning from terminal feedback a third of a point, exact knowledge of hidden state one point, and placing capital where demand persists six. The constraint is the object being routed over, and the deployable gain belongs to whoever commits scarce directional capacity in the right place.

The rest of the paper asks where that constraint goes under objects that are not bilateral channels, and what follows for a population that is not human. Section 4 types four settlement objects, the on-chain output, the channel balance, the Ark virtual output, and the optimistic escrow, as instances of one resource tuple, extends the ledger so that transfers between them are clocked settlement events, and derives the bound that replaces the cut bound under a server-mediated object. It pools: the directional forecast per edge becomes a forecast of aggregate volume per server, plus a directional term on the server's own channels that the pooling argument has to charge or it proves too much. A liquidity-duration ratio states when pooling pays, and a reference model computes it on a grid of demand processes under both readings of the recovery question, of which the implementer's server, read at a commit of October 2026, takes the expiry one. Section 5 asks the same question

of an object that does not exist yet, the multiparty channel that the soft fork proposed as BIP 448 would make practical: the graph becomes a hypergraph, the cut bound is unchanged, and the pooling a server offers becomes available inside each channel without one. A fourth object, a covenant tree proposed in October 2026, enters in Section 4 beside the others; under it the bound acquires a byte-rate beside its capital term and a throughput constraint on one output. Section 5.6 then sets the three mechanisms that share an output side by side and orders the objects by what each asks of script, a ladder on which the bound is piecewise. Section 6 turns to the reward layer such a population would need if liquidity provision is to be compensated without identities, gives a claim-once registry keyed by an object's spend authority, finds the hole a nullifier leaves and the epoch rule that closes it, and sets out what verification costs on a chain that verifies no proofs itself. Section 7 asks what a population of agents under common control could do with all of this: who observes what under each object, what three different activities called coordination amount to, how much a coalition can earn from coordinating, and what changes when the coalition runs the server, uses the escrow as a commitment device, or shares its weights. Section 8 preregisters the experiments, and Section 9 says what is assumed and not discharged.

Nothing in the paper is a live measurement. The simulator results of Section 3 are from sealed preregistered campaigns on sampled public topology with synthetic hidden state; everything after is a model, with protocol parameters taken from implementer documentation at a stated date and treated as inputs; the proof system is simulated. The reference model, its tests, and the fixtures behind every table are in the repository cited as Dugan [18], at the commit given there.

2 Related work

Lightning liquidity and routing. Pickhardt and Richter [41] formulate payment delivery over hidden balances as a minimum-cost flow problem under a uniform prior on channel balance and show large gains in reliability from splitting; the same authors treat imbalance and rebalancing [40]. Both take the channel graph and its capacities as given and optimize what crosses it. Guasoni, Huberman, and Shikhelman [24] model the channel itself as a capital decision under bilateral demand, which is the decision this paper says is binding for an agent population. The Lightning Service Provider specifications [31] standardize just-in-time channel opening, which moves the inbound decision to the moment of first receipt but keeps it per channel. The prefix cut bound of Section 3 is a conservation statement of the kind that underlies max-flow min-cut [22], applied to cumulative settled crossings rather than to a single flow.

Privacy and probing. That channel balances are only nominally hidden is established by Herrera-Joancomartí et al. [25], Tikhomirov et al. [56], and Nisslmueller et al. [37]; that the network's privacy against a well-placed observer is partial is established empirically by Kappos et al. [27] and Romiti et al. [43], and Béres et al. [5] gives a traffic analysis of the economics. We use these as the observer rows for the channel object in Section 7; the contribution there is to add the rows for objects that have a server.

Ark and covenants. Ark was proposed by Keceli [28] as a server-coordinated off-chain output scheme; its current parameters are those published by its implementers [50–52] and summarized by Bitcoin Optech [7]. The implementer's own simulation [50] compares an

Ark server's liquidity requirement to an LSP's by top-up frequency and finds the comparison flips with it; Section 4 names the quantity behind that flip and adds a term. Covenant proposals would change the channel object itself. The earliest are the template commitment of Rubin [46] and the eltoo channel of Decker et al. [14], surveyed by Bitcoin Optech [8]. The current one is BIP 448 [48], a draft that bundles a Taproot-native template hash [47], signature verification on stack messages, and a push of the internal key, and whose stated purpose is rebindable signatures: LN-Symmetry channels, in which any later state replaces any earlier one and no penalty is needed [9]; the multiparty channels and factories [10] that such symmetry makes practical; and Ark variants whose rounds need no interaction [45]. Section 5 works out what that does to the bound. LayerTree [39], a draft of October 2026, is a covenant shared output of a different design: a Merkle-sum tree committed to one output, every change enforced by tapscript, no pre-signed interior, no expiry of the output, a unilateral exit of one transaction and $O(D)$ bytes, in four opcode tiers none of which is active, and vouchers in place of Ark's out-of-round transfers. Sections 4 and 5.6 place it. The results of Sections 3 and 4 depend on nothing in it.

Sybil bounds and nullifiers. The Sybil problem is Douceur [15]'s: without a trusted authority, identities are free. Babaioff et al. [2] give Sybil-proof reward schemes for a propagation setting; the registry of Section 6 is instead a proof-of-exposure scheme, in which a claimant proves control of a settled object without revealing which. The nullifier it uses is Zerocash's [4, 26], transplanted from spend-once to claim-once, and the gap we find in the transplant is that spending is the one thing a nullifier is designed to allow.

Optimistic verification on Bitcoin. Bitcoin consensus verifies no succinct proofs. BitVM [32, 33] and BitVM3 [34] make a statement enforceable by an assert-and-disprove game under a committee-at-setup and one-honest-challenger assumption; we use the published BitVM3 cost figures as inputs to the escrow object and to the economics of Section 6. Groth16 [23] is the proof system those figures assume.

Covert channels, wardens, and collusion. The warden is Simmons [54]'s, and the capacity-versus-detectability framing of a steganographic channel is Cachin [11]'s; Section 7 argues the capacity question is second-order for this setting. Wash-trade detection [13, 57] supplies the netting statistics whose failure modes the reference model reproduces. On what a population of agents can do without a message, Calvano et al. [12] show algorithmic pricers converging on supra-competitive prices without communication; Stigler [55] located cartel instability in the unenforceability of side-agreements, which is the lever an enforceable escrow pulls; and the focal-point account of coordination without communication is Schelling [49]'s.

3 Lightning as a graph: the directional capital bound

3.1 Setting

Let $G = (N, E)$ be a directed multigraph. Each channel $i \in E$ joins two nodes u_i, v_i and has capacity χ_i, split at any time into balances $\ell_i(u_i), \ell_i(v_i) \geq 0$ with $\ell_i(u_i) + \ell_i(v_i) = \chi_i$. A node can spend over channel i only from its own side. The balances are hidden from all but the two endpoints; the capacities are public through gossip [30]. A payment of amount a from s to t is an attempt along a path; it settles only if every edge on the path

has $\ell_i(\text{upstream}) \geq a$ at the moment of the attempt, in which case a moves from the upstream to the downstream side of every edge on the path, and otherwise it fails and nothing moves. Fees are absorbed into a. Capacity changes by funding, splicing, leasing, and closing, each an on-chain event with a confirmation clock.

For a node set $A \subseteq N$, write ∂A for the channels with exactly one endpoint in A. After n settled events, let $O_A(n)$ and $I_A(n)$ be the cumulative amounts that have crossed ∂A outward and inward, counting each crossing of a path that leaves and re-enters; let $S_A(n) = \sum_{i \in \partial A} \ell_i(A\text{-side endpoint})$ be A's outbound stock on the cut and $S'_A(n) = \sum_{i \in \partial A} \chi_i - S_A(n)$ its inbound stock; and let $K_A(n)$ be the net stock added to S_A by capacity events, with $K'_A(n)$ the same for S'_A.

3.2 The cut bound

Lemma 1 (prefix cut bound)**.** *For every $A \subseteq N$ and every n,*

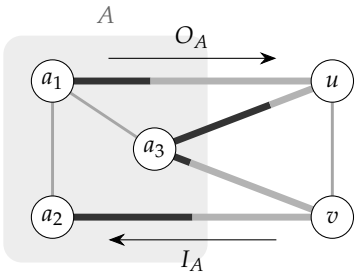
$$O_A(n) - I_A(n) \;\leq\; S_A(0) + K_A(n), \qquad I_A(n) - O_A(n) \;\leq\; S'_A(0) + K'_A(n).$$

Proof. A settled crossing of channel $i \in \partial A$ outward moves its amount from the A-side balance to the other side, decreasing S_A by that amount and increasing S'_A by the same; an inward crossing does the reverse; a capacity event changes S_A by its contribution to K_A. Events inside A or outside A touch no channel in ∂A. So $S_A(n) = S_A(0) - O_A(n) + I_A(n) + K_A(n)$, and $S_A(n) \geq 0$ because balances are nonnegative. The second inequality is the same identity on S'_A. □

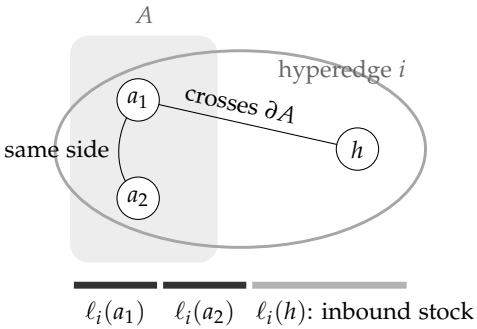
The inequalities are prefix conditions: they must hold at every n, not only at the horizon, and they bind on whichever stock is being consumed. A population of agents that mostly pays outward binds on the first with I_A small, so on K_A, which is to say on someone having funded outbound from A. A population that mostly gets paid binds on the second, on K'_A, which is inbound toward A and must be put there by A's counterparties, from their side, by a forecast. Figure 1(a) draws the cut.

Proposition 1 (policy invariance)**.** *Lemma 1 holds under every routing policy, including a policy with exact knowledge of all balances. A policy affects which attempts settle, not the identity that settled crossings satisfy. Consequently the advantage of balance knowledge over a policy of bounded retries is confined to the attempt budget: it converts failed attempts into settled ones within the same cut stock, and cannot raise net delivery across any cut above the lemma's bound.*

Proof. The lemma's proof uses only that settled crossings conserve balance on each channel and that balances are nonnegative; neither depends on how attempts are chosen. For the consequence, fix a cut A and a horizon. Both policies produce sequences of settled crossings and both sequences satisfy the bound, so neither delivers net across ∂A more than the stock $S'_A(0) + K'_A(n)$. The oracle's knowledge changes which attempts settle, and therefore the order in which that stock is drawn down, but not the stock. The two policies need not reach the same states, since an attempt that settles under one and fails under the other changes the balances every later attempt sees, so nothing here says bounded retries reproduce the oracle's sequence; what it says is that the oracle's advantage at a finite attempt budget is confined to attempts the retry policy spends failing against a stock the lemma fixes for both, which is the quantity Section 3.4 measures. □



(a) channels on ∂A. Dark: the A-side balance; light: the far side. S_A sums the dark parts, S'_A the light.



(b) the same cut through a hyperedge. χ_i is divided among the members; a transfer within one side moves nothing across ∂A.

Figure 1: The prefix cut bound. (a) On a graph, each channel crossing ∂A is one capacity split between its two endpoints. The A-side balances sum to the outbound stock S_A and the far-side balances to the inbound stock S'_A; a settled crossing moves value from one sum to the other, so $S_A(n) = S_A(0) - O_A(n) + I_A(n) + K_A(n) \geq 0$ under any routing policy (Lemma 1). (b) On a hypergraph a channel's capacity is divided among all its members. A crossing changes the two sums as before, and a transfer between two members on the same side changes neither, so the bound is unchanged (Lemma 2); for the agents inside A the inbound stock on channel i is the hub's part alone, one number for the group.

3.3 The provisioning problem has no pooling

The quantity that Lemma 1 makes the operator choose is K'_A: how much inbound to put toward each agent a before a's demand is known. Let D_a be a's demand process over a horizon H, and let $M_a = \max_{t \leq H} \left(\text{receipts}_a(t) - \text{spends}_a(t)\right)^+$ be the peak balance a will hold in the channel. A receipt that would push a's balance above the provisioned inbound fails. The operator chooses $\hat{f}_a$ before seeing D_a and pays for idle capital if $\hat{f}_a > M_a$ and for failed receipts if $\hat{f}_a < M_a$: a newsvendor problem [1], one per channel.

What makes the problem expensive for a population is not that each instance is hard but that there are N of them and they do not pool. Capacity provisioned toward agent a sits on channel a and serves no other agent, so the total inbound an operator must carry is $\sum_a \hat{f}_a$, a sum of forecast peaks, regardless of how the agents' demands are correlated. If the same N agents drew on a common pool, the operator would carry a forecast of the peak of the sum, $\hat{f}_S$, and for independent imbalances the two differ by a factor that the following remark sizes.

Remark 1 (scale of the pooling gain). Let each agent's per-round net inflow be independent with mean μ and variance σ^2. The agent's cumulative imbalance is a random walk whose running maximum over T rounds has expectation of order $\sigma\sqrt{T}$ when $\mu = 0$ and order μT when $\mu > 0$ dominates. The sum of N such walks is a walk with variance $N\sigma^2$ per round, so its running maximum is of order $\sigma\sqrt{NT}$ at zero drift and $N\mu T$ otherwise. The ratio of sum-of-peaks to peak-of-sum is therefore of order $\sqrt{N}$ at zero drift and of order 1 when a common drift dominates, since each agent's peak and the sum's peak are then both set by accumulated drift and both sides come to $N\mu T$. That is the scale for imbalances that can take either sign, which requires agents holding balances they can spend down. An agent that starts from nothing cannot spend what it has not received, and its balance is the walk reflected at zero. Reflected walks do not net. The sum of N of them is N times the

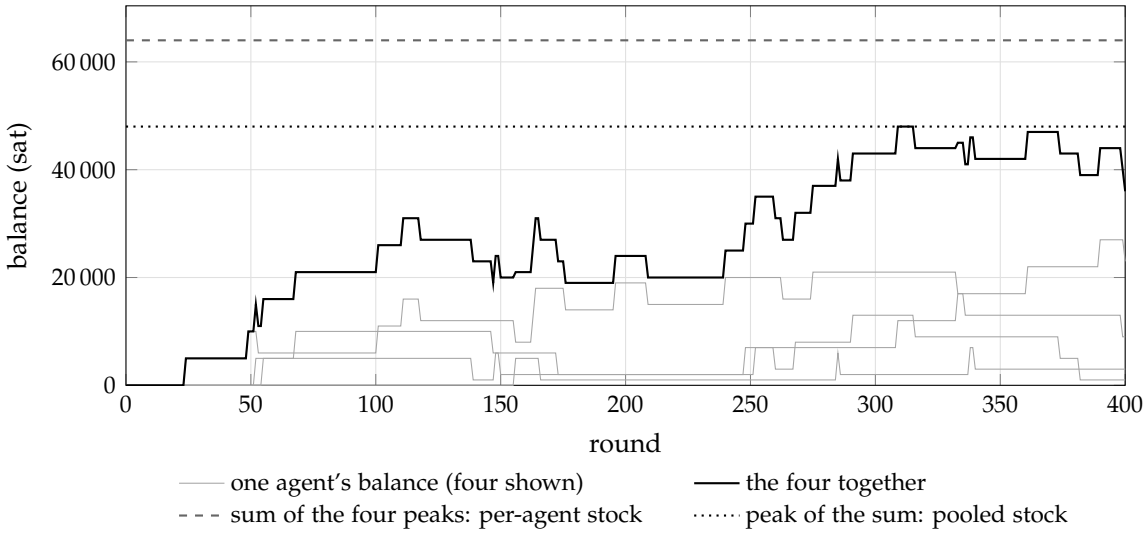


Figure 2: Four agents of the zero-drift cell, drawn with the model's generator over 400 rounds. Each balance is a random walk reflected at zero, since an agent spends only what it has received. Per-agent provisioning must hold each agent's own peak, and the sum of those peaks is the dashed line; a pool must hold the peak of the sum, the dotted line. The gap between the two lines is the whole pooling gain. It is a constant factor and not $\sqrt{N}$, because the reflected walks all sit above zero and never cancel one another (Remark 1). Generated by `model/grid.ts`.

expectation of one, up to fluctuations of order $\sqrt{N}$, and since that expectation grows with time the peak of the sum is at least $N\sigma\sqrt{2T/\pi}$, against a sum of peaks of $N\sigma\sqrt{\pi T/2}$ in the diffusion limit. At zero drift the ratio therefore rises from 1 toward $\pi/2$ as N grows and does not pass it. Under a draining drift each balance is stationary and the ratio is that of its peak to its mean, a larger constant that grows slowly with T; under an accumulating drift it is 1 as before. Pooling buys a factor of $\sqrt{N}$ on signed imbalance, a constant near $\pi/2$ on the imbalance of agents with nothing to spend down, and nothing on correlated accumulation.

That remark is the whole of what Section 4 finds when it computes the thing exactly, and it is worth having in view before the objects are introduced: a settlement object relocates the forecast from per-edge to per-pool, and the relocation is worth $\sqrt{N}$ on one kind of demand, a constant on the kind this paper's population presents, and nothing on a third. Figure 2 draws the comparison on four of the model's agents, and Table 1 checks the three regimes on the model's demand law: the signed ratio grows as $\sqrt{N}$, the reflected one saturates at 1.55 by $N = 200$, and a shared drift holds it at one.

3.4 What a simulator shows

Two sealed, preregistered campaigns on sampled public Lightning topology with synthetic hidden balances [18] measured the quantities above from the routing side. The topology was sampled from gossip snapshots; hidden balances were drawn per channel; demand was a stationary process with a predeclared shift in one arm; outcomes were cluster-robust over sampled topologies, with a practical-equivalence band declared in advance. Table 2 gives the results.

Read against the lemma, the table says what it should. Everything a routing policy

N	reflected	signed	signed$/\sqrt{N}$	shared drift
1	1.00	1.00	1.00	1.00
2	1.14	1.50	1.06	1.00
4	1.27	2.07	1.04	1.01
8	1.34	2.94	1.04	1.01
16	1.42	3.80	0.95	1.01
32	1.46	4.81	0.85	1.01
64	1.51	7.07	0.88	1.01
128	1.52	11.82	1.04	1.01
200	1.55	15.71	1.11	1.01

Table 1: The scale of the pooling gain on the model's own demand law: the sum of the N agents' peak balances over the peak of their sum, over 5,376 rounds, averaged over 20 sample paths. Reflected is the model's agents, who spend only what they hold, and the ratio saturates near $\pi/2 \approx 1.57$. Signed is the same increments with balances allowed either sign, and the ratio grows as $\sqrt{N}$, so the third column stays near one. Shared drift is $+60$ sat a round for every agent, the grid's drift cell: nothing nets and the pool holds the sum of the peaks. Generated by `model/grid.ts` from `model/pooling.ts`.

Intervention	Gain, success points	Reading
Bounded retries over one-shot routing	$+5.14$	attempt budget
Learning from terminal feedback, over retries	$+0.35$	inside equivalence band
Exact hidden-state knowledge, over retries	$+1.11$	attempt budget, not information
Failure-aware placement over random, stationary demand	$+6.13$ (4.91 to 7.35)	capital
Failure-aware placement over random, after demand shift	≈ 0	capital, mis-forecast

Table 2: Simulator results from the preregistered public-topology campaigns [18]. Success points are percentage points of settled payments at a fixed attempt budget. Under free failures the oracle's advantage was shown to be an attempt budget rather than information, which is Proposition 1 observed.

can do is worth about one point over a wallet that retries, because the policy is choosing attempts within a stock it cannot change. Placing the stock where demand persists is worth six, and the six vanish when demand moves, because the placement was a forecast and the forecast was wrong. The gain is in K'_A, and K'_A is a forecast made per edge.

3.5 The ghost-node apparatus

The campaigns used a graph-rewrite construction from earlier work in a clearing setting [19]: augment the graph with a nonphysical node or edge, solve a balancing problem in the augmented graph, and map the solution back to permissible operations, here to funding, splicing, or leasing on a real edge. In the channel setting the construction is a solver for K'_A: a ghost edge is a unit of inbound the solver wishes existed, and compiling it is the act of forecasting. Section 4 will move that solver one tier up.

4 Ark: where the bound moves

4.1 Settlement objects as typed resources

The channel is one way to hold value off-chain. To compare it with others on the terms the lemma cares about, we need each object described by what it costs to exit, who can hurt its holder, what the holder must do to keep it safe, and when its status changes.

Definition 1 (settlement object)**.** A settlement object is a tuple $\rho = (\mathcal{A}, v, F, X, \Lambda, \Gamma, \tau)$: a spend authority $\mathcal{A}$ (a script or a signer set), a value v, a finality clock F mapping elapsed time or block height to a settled or pending status, a unilateral exit X with a cost and a delay, a liveness obligation Λ the holder carries to keep the object trustless, a set Γ of parties whose misbehavior can cost the holder value or access, and an expiry τ after which the object's trust assumptions change.

Table 3 instantiates the tuple for the on-chain output U, the directional channel balance C, the Ark virtual output V, and the optimistic escrow E, with a multiparty channel C_k and a native-covenant escrow E' for comparison, neither available today. The Ark parameters are those published by Second as of September 2026 [51, 52]: a virtual output (VTXO) is an off-chain output in a tree whose root is an on-chain round transaction; it is spendable by the holder and server together, or by the holder alone after a relative timelock, by broadcasting its branch; it expires after a server-configurable lifetime, 28 days by default, before which the holder must refresh it in a round; and an out-of-round transfer is a pending state the recipient trusts until the next round confirms [7].

Three things the table makes visible. Of the objects available today, the channel is the only one whose value is direction-partitioned: a VTXO of value v is spendable to any counterparty the server will co-sign for, so there is no inbound side to provision at the holder's tier. The VTXO and the LayerTree leaf are the objects with an obligation on the holder's calendar, with a difference: the VTXO's holder refreshes or forfeits to the server, the leaf's renews or is paid out on chain. Either obligation is one an always-online process meets at no marginal cost, as it also meets the channel's breach-watching; liveness distinguishes agents from humans, not one object from another. And the escrow is the only object whose finality is a game rather than a count, with a committee rather than a counterparty in Γ.

The LayerTree leaf. The fourth object is a draft and not a deployment [39]. Its state is a leaf, an amount with an owner key, an exit key, an evictor key, a lease and a counter, at a position in an off-chain Merkle-sum tree of depth D; the output commits to the root through its internal key, the predicates are its taptree, and every spend proves the changed positions against the committed root and recreates the output under the new one. Authorisation is the draft's Table 4: the owner exits at any time, co-signs updates, issues vouchers and carves its leaf; the evictor pays a leaf out in full after its lease; the operator grafts, co-signs, evicts, detaches a cohort and fills empty positions, and cannot move or reduce another leaf, delay an exit beyond the draft's assumption [C] (Section 4.7), or freeze value it did not graft and fund; anyone redeems a valid voucher. The exit is one transaction and $O(D)$ bytes with no timelock: 1,020 vB at $D = 16$ in the FULL tier on the draft's transaction model (1,030 serialised), 762 to 1,149 vB across $D = 8$ to 20, 1,355 in MID, 1,473 in MIN, 908 in CCV; the owner must be online to build it and to rebuild it after a competing spend, unless a tower holds its EXIT^t authorisation (1,046 vB). The output does not expire; each leaf carries a lease after which its evictor may pay it out on chain, 158 to 203 vB

	$\mathcal{A}$	F	X	Λ	Γ	τ
U on-chain output	script	k confirmations	none	none	consensus only	∞
C channel balance	2-of-2 + HTLC	HTLC settle, bilateral	commitment tx + `to_self_delay`	watch for breach	counterparty liveness	∞, direction-locked
V Ark VTXO	holder $\wedge$ server (MuSig2), or holder after CSV	round confirmation; out-of-round pending	branch + leaf broadcast, CSV 144 ($\approx$24 h)	refresh before τ	server (censor); sender + server (collude on pending)	28 d, server-set
E optimistic escrow	release after Δ, or slash on disprove	challenge period Δ	on-chain already	≥ 1 honest challenger online	setup committee (1-of-n deletes key); proof-system setup	program-defined
C_k multiparty channel	n-of-n of members (MuSig2), rebindable	update signed by all members	latest state, then settle after CSV	every member signs each update; watch for stale state	every co-member (availability)	∞
T LayerTree leaf	owner key; Merkle-sum path to the root in the output key	the spend confirms; no timelock	one tx, $O(D)$ bytes, 1,020 vB at $D = 16$; owner online, or a tower	renew the lease or be paid out; a fee coin for most exits	operator (graft, co-sign, evict after lease, paying in full; fill the empty positions a partial voucher needs); a voucher's issuer until confirmed	none on the output; lease per leaf, eviction pays the member
E' covenant escrow	template-enforced script	template match	on-chain already	none	consensus only	program-defined

Table 3: Four settlement objects as instances of the tuple, with three for comparison that are not available: the multiparty channel C_k of Section 5, of which C is the case $k = 1$, the leaf T of the LayerTree covenant tree [39], and a native-covenant escrow E'. No covenant or rebinding opcode is active on mainnet at the cited date [8]; BIP 448 [48] is the current proposal for C_k and E', and T needs more than it supplies. MuSig2 is Nick et al. [35].

per leaf in a cohort sweep, and keeping a position costs a renewal per period, 151 vB per member in a signed cohort rewrite or 2,616 vB alone. The covenant dependency is one of four opcode tiers: FULL (`OP_CAT`, splice and bit operations, `OP_TX`, CSFS, the EC opcodes), MID (`OP_CAT`, `OP_TX`, CSFS, `OP_TWEAKADD`; four-byte numerics cap a tree at 2^{31} sat), MIN (`OP_CAT`, `OP_TWEAKADD`; exits and splits only, so no vouchers and no channel leaves) and CCV (BIP-443 and `OP_TX`). None is active, three assume an opcode in a tapleaf version its proposal does not define, and BIP 448 supplies the CSFS that MID uses in the version BIP 348 defines and that FULL, and CCV when it hosts vouchers, assume in another; nothing else a tier needs is in BIP 448. Every figure this paper quotes for T is on the draft's model, which prices a whole transaction on a 946 weight-unit skeleton, rounds once, and omits the witness length prefixes (about one percent in FULL and CCV, three in MID and MIN) and, in its baselines, anchors and fee children.

4.2 The extended ledger

Let the state at step n be

$$X_n = (\ell^U, \ell^C, \ell^V, \ell^E,\ h,\ \mathcal{U},\ \mathcal{J},\ \kappa_n),$$

with $\ell^{\cdot}$ the holdings vector for each class, h the pending holds, $\mathcal{U}$ the registry of Section 6, $\mathcal{J}$ the journal, and κ_n the vector of clocks: block height and per-object elapsed time. The per-channel constraint $\ell^C_{i,+} + \ell^C_{i,-} = \chi_i$ is retained for class C only; class V carries a per-holder constraint $\ell^V_u \geq 0$ and the per-server liability of the next subsection; class E carries a per-escrow state.

A cross-class transfer is a settlement event $\sigma = (\text{from}, \text{to}, \text{amount}, \text{clock condition})$, journaled at initiation, held in h, and moving ℓ only when its clock fires: $U \to C$ is a funding transaction with condition "k confirmations"; $V \to C$ is an HTLC-scripted VTXO the server spends into a Lightning HTLC, with condition "HTLC settled"; $V \to U$ is an offboard or a unilateral exit with condition "CSV elapsed"; $U \to E$ is an escrow deposit; $E \to U$ is release after Δ or slash on disprove.

Proposition 2 (cross-class conservation)**.** *Let a finite set of settlement events be journaled and let every event's clock condition have fired or the event have been aborted. Then the sum over classes and holders of ℓ, plus fees paid to relays and miners, plus amounts slashed, equals its initial value plus external funding. During pending intervals the identity holds with h included on both sides.*

The proof is bookkeeping: each event debits a hold on its source at initiation, and at settlement releases the hold, debits the source, and credits the destination or the slash account; an abort releases the hold and moves nothing. What the proposition rules out is counting a declared cross-system claim as liquidity before its clock has fired, which a ledger without clocks cannot prevent. What it does not establish is that any clock fires: a round may not confirm, an HTLC may time out, a challenge may end in a slash. Those are per-object finality assumptions and are inputs.

4.3 The server bound

Let a server S issue VTXOs to a holder set A. Holders carry no directional stock. The server's liquidity is a stock R_S with three sinks and two sources per round [50]: it funds new outputs at refresh, fronts Lightning spends at the moment of spend, and funds offboards; it recovers forfeited or expired value by sweeping a round after its absolute timelock, and it receives inbound HTLC settlements on its own channels. Write $V_A(n)$ for the

VTXO value held by A, $W_S(n)$ for value the server has fronted into rounds and not yet recovered, and B_S for the server's total on-chain and channel capital.

Proposition 3 (bound relocation and instantaneous server headroom)**.** *For a set of VTXO holders A, count $O_A(n)$ and $I_A(n)$ as the face value of* settled *outgoing and incoming transfers through step n. Let $V_A(0)$ be their initial settled balance, and let $J_A(n)$ denote any separately recorded net external issuance into A that is absent from $I_A(n)$, with fees and burns included among outflows. Nonnegative holder balances imply*

$$O_A(n) - I_A(n) \;\le\; V_A(0) + J_A(n).$$

This is the holder-stock identity; in the reference model, receipts recorded in I_A already create VTXOs, and those receipts must not also be added to J_A.

For the server, let B_S be the fixed capital budget available for fronting*, let $W_S(t^-)$ be gross fronted value awaiting recovery immediately before an admitted event at time t, and let $F_A^S(t)$ be new gross input value fronted at that event for holder set A. If $R_S(t)$ is the gross value subsequently released by confirmed sweeps, the admission condition and outstanding-capital invariant are*

$$\sum_A F_A^S(t) \;\le\; B_S - W_S(t^-), \qquad W_S(t^+) = W_S(t^-) + \sum_A F_A^S(t) \;\le\; B_S,$$

with a sweep reducing W_S by $R_S(t)$ when that recovery becomes available. The A in the sum partitions the gross inputs of one atomic event, so the same input is counted once. These constraints apply to current outstanding commitments; the cumulative value fronted across the entire horizon may exceed B_S through reuse after recovery.

For the holder identity, rearrange the settled balance equation $V_A(n) = V_A(0) + I_A(n) + J_A(n) - O_A(n) \geq 0$. For the server invariant, proceed by induction over admitted fronting and confirmed recovery events: the admission guard bounds each increase, and each recovery lowers the outstanding stock. A pending payment and an unconfirmed sweep are excluded from settled balances and available recovery respectively. A concrete reference-model check uses $B_S = 1{,}300{,}000$ sat, three 300,000-sat payments funded by 400,000-sat gross inputs each, and $W_S = 1{,}200{,}000$ sat. The current headroom is then 100,000 sat while the cumulative outbound payments are 900,000 sat, a counterexample to imposing the headroom bound on *cumulative* allocations. In `model/server.ts`, `aggregateBoundHolds()` checks $W_S \leq B_S$; the `allocated` map accumulates across time and is not the $F_A^S(t)$ of this proposition.

These are fronting-budget constraints under the reference model's admission and recovery rules. Issued VTXO liabilities, server insolvency, and actual Ark recovery semantics require additional protocol accounting; the simulator's `receive` operation accepts external issuance without independently proving that the server has provisioned its backing. The holder must refresh before τ, the server can decline to include a holder in a round, and an out-of-round payment remains pending until the round confirms. Under the LayerTree leaf the holder renews a lease or is paid out, and the operator can decline a graft or co-signed update while a valid unilateral exit has a separate path.

What the proposition does not settle is the forecast at the server tier. Each Lightning receipt into an Ark produces value for a holder while consuming capacity on the server's own Lightning channels. The aggregate demand forecast $\hat{f}_S$ is the peak-of-sum of Remark 1; pooling replaces individually provisioned $\sum_a \hat{f}_a$ with that aggregate stock under the stated demand assumptions. Under signed uncorrelated imbalances the theoretical gain can scale as $\sqrt{N}$, while the reflected-at-zero case considered in the reference model

gives a constant near $\pi/2$ in its diffusion limit. Shared drift eliminates that pooling gain. The server is also a node in G, and Lemma 1 applies to the cut $\{S\}$. Receipts cross $\partial\{S\}$ and consume directional capacity that counterparties must provision. That network-facing stock is common to the compared settlement objects and is handled by the single-tier accounting convention in Section 5.4; boarding from an on-chain UTXO follows a different path.

What the implementation does. The two readings can be read off the implementer's server [53], and the deployed one is the expiry reading. A Lightning spend leaves the server holding the payer's HTLC output inside the payer's round. The server's watchman sweeps server-owned outputs on chain only once their round has expired (`server/src/watchman/policy.r` the server-owned and expiry policies), and a round's outputs may not carry an HTLC policy (`server/src/round/mod.rs`), so the value is not carried into the next round either; the implementation does not transfer a received HTLC output onward off chain. A Lightning receipt is not issued from that value. It is issued out of a pool of server-owned outputs (`server/src/vtxopool.rs`, `server/src/ln/mod.rs`) that the server funds on chain from its rounds wallet as a signed tree of its own leaves, in buckets of a configured amount and count, re-issued when a bucket falls below a threshold, 80 percent by default, with a default lifetime of 432 blocks and discard 144 blocks before expiry; only an issuance and the change of a pool spend ever enter the pool. So the implementation carries the forfeit lock of equation (1) and adds, in concrete form, the second stock the model prices as the server-tier term: an inventory of issuance capital sized in advance per server and locked for its lifetime whether or not it is used. The bucket targets are operator configuration and the production values are not published, so the model keeps $\hat{f}_S$ as a forecast of the aggregate and does not fit the inventory.

4.4 The ratio that decides

Definition 2 (liquidity duration)**.** For a demand process D over a horizon H, the liquidity duration of an object is locked capital-time per unit delivered,

$$\mathcal{D}_{\text{obj}}(D, H) = \frac{1}{\text{Vol}(D, H)} \int_0^H \text{Locked}_{\text{obj}}(t)\, dt,$$

with Vol counting delivered volume in both directions.

For the channel, $\text{Locked}_C(t) = \sum_a \hat{f}_a$, held for the horizon whether or not the peak is reached. For the VTXO, $\text{Locked}_V(t) = W_S(t) + \hat{f}_S$: the fronted-and-unswept value, each unit locked from the moment of spend or refresh until it is recoverable, plus the server's own pooled stock toward its holders, $\hat{f}_S$. Writing $\bar{L}$ for the mean residual lock at forfeit,

$$\frac{\mathcal{D}_C}{\mathcal{D}_V} \;\approx\; \frac{H \sum_a \hat{f}_a}{\text{Vol} \cdot \bar{L} \;+\; H\hat{f}_S}. \tag{1}$$

The ratio exceeds one when pre-funded per-agent capacity sits idle for long relative to what the server locks: many agents, idiosyncratic and uncorrelated imbalance, a horizon long against $\bar{L}$. It falls below one when balances recycle within one channel faster than $\bar{L}$, so that a small envelope serves a large volume: a few agents in steady bidirectional flow with fixed counterparties. An agent population is not favored by the VTXO because it is high-frequency; high frequency to a fixed counterparty is the channel's best case. It is favored when the population is large and its imbalances net across members.

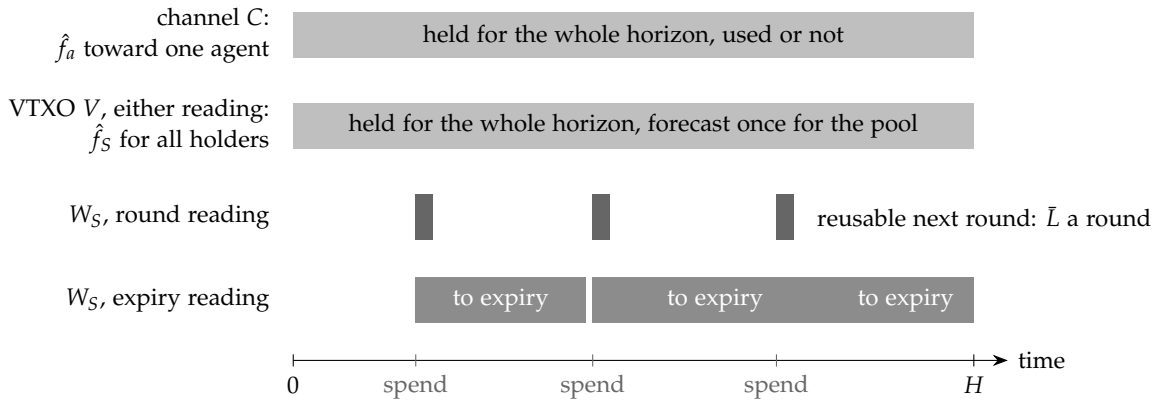


Figure 3: Locked capital-time, the quantity the liquidity duration (Definition 2) divides by delivered volume, drawn as bars whose area is capital times time. A channel's stock toward an agent is held for the horizon whether or not the agent's peak arrives. Under either reading the server's pooled stock $\hat{f}_S$ is held likewise, forecast once for all holders. What differs between the readings is the forfeit lock W_S: value the server fronts at a spend is reusable one round later under the round reading, or only when the spent output would have expired under the expiry reading, so the mean residual lock $\bar{L}$ is a round in one case and about half a lifetime in the other; the third bar runs past the horizon. The implementer's documentation does not settle which reading holds; Table 4 carries both, and the first preregistered item is to pin it.

Equation (1) has one input that the cited documentation does not fix, and it sets the scale of $\bar{L}$. When the server fronts a Lightning spend from a holder's VTXO, the forfeited input is either swept only at its absolute expiry, in which case $\bar{L}$ is on the order of half a lifetime, or presented to the next round as an input and its value reused one round later, in which case $\bar{L}$ is one round interval. The implementer states that Lightning payments use HTLC-based policies rather than forfeits [51], which is consistent with either reading. The reference model carries both as a parameter. Figure 3 draws the two readings beside the channel's lock.

The LayerTree regime. Under T nothing is fronted at a spend. A member pays by voucher, a signature over a leaf that its holder redeems alone, or in a channel leaf with capital of its own, so W_S has no counterpart, and what the operator fronts comes before the receipt. A member receives over Lightning into a credit leaf the operator owns, with the member's key as exit key: the operator settles the payer's HTLC first and then signs the member a cumulative voucher for the running total, and since a voucher draws only what the leaf holds, the operator pre-funds the leaf and tops it up at about 122 vB per leaf in a cohort rewrite. The pre-funded capital P_a is the operator's, bound from top-up until the member co-signs a settlement, and on every path the operator can take alone its excess is the member's, so the forfeit runs the opposite way to Ark's and the operator's incentive is to fund little and top up often; the member's exposure is the one payment settled and not yet vouched for. A pending-leaf variant, with a reclaim key and a landing leaf per member, removes that exposure and bounds the operator's lock at a settle window W of its choosing. Two further terms are bytes rather than capital. A graft admits a cohort at under 3 vB per member when the operator funds it, fronting the cohort's value G until members pay, or nothing if they prepay by voucher, and at 58 vB more per member when

members fund it, which is then a signing round among them; and a lease is renewed at 151 vB per member per period T_ℓ in a cohort, or 2,616 vB alone. With the convention of Section 5.4,

$$\text{Locked}_T(t) = \sum_a P_a(t) + G(t), \qquad P_a \approx (1+m)\, r_a\, T_u, \tag{2}$$

$$\frac{\mathcal{D}_C}{\mathcal{D}_T} \;\approx\; \frac{H \sum_a \hat{f}_a}{H \sum_a (1+m)\, r_a\, T_u \;+\; \int_0^H G\, dt}, \tag{3}$$

where r_a is the member's receipt rate and T_u the top-up period or the settle window, and beside it a byte-rate the duration does not price, $b_T = 122/T_u + 151/T_\ell$ vB per member per block in cohorts of 64. The credit leaf converts the channel's long-horizon forecast of a peak net balance into a short-horizon forecast of gross receipts, per member and unpooled, and pays for the shorter horizon on chain. On the zero-drift cell of Table 4, receipts of 100 sat a round, a daily top-up at margin 0.25 sizes P_a at 3,000 sat against about 104,000 sat of channel stock per agent under the oracle rule and about 38,000 under the trailing rule, some thirty-five and thirteen times less, for 0.88 vB per member per block. The bytes are not free, and the exchange rate between them and capital is a cost of capital and a fee rate, which the comparison needs and the duration does not contain. At 2 percent a year in bitcoin terms, the oracle saving carries about 2,000 sat a year, and the byte-rate is about 46,500 vB a year per member, 93,000 sat at 2 sat/vB: on this cell the bytes cost forty-six times the carry at 2 sat/vB, and the leaf pays only above a cost of capital near 90 percent a year. The rate is a bitcoin rate and not a fiat lending rate, since the capital here is sat held against sat: the reference rate of Bhatia [6], the annualised routing yield on channel capital, is its floor, with examples near a quarter of a percent, and the liquidity-lease markets of mid-2026 are its observable, from half a percent on liquidity ads to four on the smallest Magma channels [3]. The bytes are fixed per member and the saving grows with receipts, so the leaf breaks even at receipts of about 4,600, 23,000 and 115,000 sat a round at 2, 10 and 50 sat/vB against the oracle stock, about two bitcoin a year of receipts at 10 sat/vB, and about three times those against the trailing; the break-even scales as the fee rate over the cost of capital, so at the reference rate's quarter of a percent it is eight times higher. The grid cannot confirm these figures, since its agents net receipts against spends in one balance and a credit leaf is receive-only; they are given as indicative. At MIN there is no credit leaf and no receive term, and no transfer that is not a chain transaction (Section 5.6). The parameters added are D and the tier, T_u, T_ℓ, W, the funding mode and G, the receive share of demand, and the κ and β of Section 4.7.

4.5 Reference-model results

The reference model [18] implements the server liquidity ledger with the forfeit-to-sweep rule under both recovery readings, the holder refresh cycle, the server-tier term computed as the channel model applied to the server's aggregate flow, and the channel model under two forecast rules: an oracle rule that sets $\hat{f}_a$ to the running peak over the whole horizon, with a margin, and holds it throughout, so that no receipt fails and the channel is charged for the idle capital of a provisioning that is never revised; and a trailing rule that re-provisions each lifetime from the previous lifetime's peak, which locks less and fails the receipts the previous lifetime did not predict. The liquidity duration charges locked capital only: a failed receipt is not delivered and so does not enter Vol, but it carries no price of its own, so part of what the trailing rule saves in $\mathcal{D}_C$ it pays in failures the ratio does not show. Table 4 gives $\mathcal{D}_C/\mathcal{D}_V$ on a grid of demand processes; Table 6 gives the failed events

	Swept at expiry		Reused next round	
Demand cell	oracle	trailing	oracle	trailing
200 agents, bursty, drift 0, $H = 8\tau$	0.85	0.46	1.46	1.06
200 agents, bursty, drift 0, $H = 1\tau$	0.63	0.63	1.40	1.40
200 agents, bursty, drift +60, $H = 8\tau$	0.97	1.05	0.99	1.11
200 agents, bursty, drift −30, $H = 8\tau$	0.63	0.33	2.52	1.52
3 agents, steady, drift 0, $H = 8\tau$	0.01	0.00	0.71	0.88

Table 4: $\mathcal{D}_C / \mathcal{D}_V$ on the reference model, server-tier term charged, two-way volume, margin 0.25, lifetime $\tau = 4032$ blocks, hourly rounds, refresh two days before expiry. Drift is expected net inflow per agent per round in sat; bursty cells draw inflows of 5000 and outflows of 4000 at rates set by the drift; the steady cell draws 100–110 each way every round. At $H = 1\tau$ the trailing rule has one interval and coincides with the oracle. The oracle column charges the channel for a whole-horizon peak held throughout and fails nothing; the trailing column charges less capital and fails what the previous lifetime did not predict, and those failures are not in the ratio (Table 6). Values above one favor the virtual output. Generated by `model/grid.ts`.

Accounting, drift +60 cell, expiry lock, oracle forecast	$\mathcal{D}_C / \mathcal{D}_V$
no server-tier term, outbound-only volume	7.14
no server-tier term, two-way volume	25.42
server-tier term charged, two-way volume	0.97

Table 5: The same cell under three accountings. Dropping the server-tier term, or counting only outbound volume on the VTXO side against two-way volume on the channel side, manufactures a large ratio on a cell where all agents accumulate together and pooling has nothing to pool.

behind each row; Table 5 shows what the ratio does on one cell when the server-tier term or the two-way volume convention is dropped.

Under the expiry reading the virtual output is not favored: the ratio runs from near zero on few-steady to about one on correlated drift (0.97 oracle, 1.05 trailing), where the server's own channels carry the sum of the agents' peaks and the forfeit lock adds to it. Under the round reading it is favored on the many-agent cells whose imbalances are uncorrelated, by a factor between 1.40 and 2.52 under the oracle forecast and between 1.06 and 1.52 under the trailing forecast; by a tenth or nothing (1.11, 0.99) on the cell where all agents drift together; and not at all on few-steady. The implementation takes the expiry reading (Section 4.3), so the left pair of columns is the deployed case; the right pair is the bound a recovery rule that reused fronted value at the next round would reach. The oracle column is the larger one because the oracle holds the whole-horizon peak for the whole horizon and fails nothing, while the trailing rule holds one lifetime's peak at a time and fails what the previous lifetime did not predict, and those failures are not priced in $\mathcal{D}_C$: on the drift-0, $H = 8\tau$ cell the trailing channel locks about a third of the oracle's capital and fails two and a half times as many events (Table 6). The two columns are an upper and a lower charge on the channel, not a forecast with and without error, and the honest range is their span. The few-steady row is weaker than its ratios look: there the trailing rule, provisioned from a lifetime of 100–110 sat flows, fails about forty percent of events and the server-tier forecast fails with it, so 0.00 and 0.88 compare two systems that are both

		Channel		Server tier	VTXO
Demand cell	events	oracle	trailing	trailing	
200 agents, bursty, drift 0, $H = 8\tau$	48 623	2 766	6 927	24	2 766
200 agents, bursty, drift 0, $H = 1\tau$	6 092	944	944	0	907
200 agents, bursty, drift $+60$, $H = 8\tau$	32 337	165	6 272	897	165
200 agents, bursty, drift -30, $H = 8\tau$	56 671	8 856	12 931	0	8 856
3 agents, steady, drift 0, $H = 8\tau$	32 256	8	12 703	8 229	4

Table 6: Failed events behind Table 4, out of the events each cell attempts; the recovery rule does not change them. A channel failure is a receipt that exceeds provisioned inbound or a spend from an empty balance; the VTXO column is the spends from an empty balance alone, which both objects fail alike, so the gap between a channel column and the VTXO column is what the channel's provisioning adds. Under the oracle rule it adds almost nothing and the channel pays in locked capital; under the trailing rule it adds a great deal and the channel pays less in capital. The server-tier column is the server's own inbound forecast failing under the trailing rule; under the oracle rule it is zero. Generated by `model/grid.ts`.

mostly refusing traffic, and the row supports the direction and not the magnitude. That is the reflected case of Remark 1. The model's agents start from nothing, so the pooled stock is a sum of balances that do not net: the oracle column sits just under $\pi/2$ at zero drift and above it under a draining drift, and $N = 200$ buys nothing like $\sqrt{200}$. The direction the ratio predicts obtains under one reading of the recovery rule and one condition on demand. The first item of the preregistration is accordingly to pin the recovery rule against the implementation. The reference model also carries the LayerTree draft's cost tables and reproduces its exit and lifetime tables on the draft's own model (Appendix B); it computes no grid cell for T, for the reason given above.

4.6 The star topology

If agents hold VTXOs and pay Lightning invoices through the server's HTLC-scripted outputs, the graph the agents see is a star with the server at its center, and the server's own channels are the only edges with direction. The cut deficits live on the server's channel set, not among the agents; the ghost solver's job is to size and place that set given the aggregate demand its holders present, and a ghost edge compiles to a connector on the server side. The apparatus of Section 3 is moved one tier up and applied to a smaller, better-observed graph, with $\hat{f}_S$ as its target. A LayerTree is a star of the same kind for Lightning receipts, which arrive on the operator's channels and are vouched into credit leaves; a voucher between two members crosses no operator and no cut the operator sees until it is redeemed. A designed channel set can also be given a shape the public graph lacks; Appendix C records one such shape, the guarantee it carries, and what the guarantee costs in the terms of this paper.

4.7 One output: throughput as a constraint

Every LayerTree operation spends the tree output and recreates it, so every member's exit, every graft and every sweep stands in one queue, and the draft states the consequence as an assumption rather than proving it away. Assumption [C]: at most $\kappa \geq 1$ spends of

a tree output confirm per block, and while an honest spend is pending at the prevailing feerate at most β conflicting adversarial spends confirm ahead of it. κ is mempool policy and not consensus: 25 under Bitcoin Core's ancestor and descendant limits before version 31 and at most 64 under its cluster limit since (`src/policy/policy.h` at v31.1 carries 25, 25 and 64), and as low as 1 once anyone hangs descendants off a payout; β is the adversary's fee budget against the honest bid; fee pinning on the output is outside [C], in the draft's words, and outside this paper. An honest spend with q others queued ahead confirms within

$$\Delta(q) = \Delta_L\left(\left\lceil \frac{\beta + q}{\kappa} \right\rceil + 1\right) \tag{4}$$

blocks, Δ_L the chain's own delay, so that ten thousand cooperating leaves leaving at once take 400 blocks at $\kappa = 25$ and 157 at 64, and a tree is sized by exit throughput and not by depth. The operator's one lever is a detach, which moves an aligned cohort into an output of its own, 780 vB for a cohort of 256 at $D = 16$, after which κ applies per output.

The constraint is not LayerTree's alone, and the table of objects should be read with it. Per-agent channels are independent outputs and have none. An Ark member's unilateral exit is D transactions on outputs that become independent as the branch is published. A multiparty channel of Section 5 is one output per hyperedge, so its updates and exits queue under the same κ, which that section's exit cost of order k should be read with. A LayerTree is κ per tree output, shardable by its operator.

The constraint also names an attack, which the draft gives as the reason for [C] and which Section 7 will want. A corrupt member can carve its own balance back and forth: each carve is one spend of the output that conserves value and costs only its fee, 1,586 vB for a split the owner signs alone, 1,618 for a claim on a voucher it issued to itself. To keep an honest 1,020 vB exit out of a block the attacker needs κ such spends confirmed ahead of it at no less than the honest bid, so its cost per block of delay is κ carves at the honest feerate: 39,650 vB at $\kappa = 25$, about thirty-nine times the honest exit's fee per block; 101 kvB at 64; 1,586 vB, 1.6 times, at $\kappa = 1$. That ratio is what β prices. It is also the attack's signature, since every transition of the output is public: a run of carves on one leaf with no payout, at a rate near κ, is visible from the spend chain alone, to anyone, with no observer inside the tree. The reference model carries $\Delta(q)$, the drain times and this cost (Appendix B).

5 BIP 448: the bound under rebindable transactions

Everything so far holds on Bitcoin as it is. This section asks what changes under one proposed soft fork, because it changes the object the lemma is about. BIP 448 [48] is a draft, assigned in March 2026 and not activated, that adds three operations to Tapscript. `OP_TEMPLATEHASH` [47] pushes a hash of the spending transaction's version, locktime, input sequences, outputs, and annex, so that an output can commit to the transaction that must spend it without committing to which coin funds it. `OP_CHECKSIGFROMSTACK`, abbreviated CSFS below, verifies a signature on a stack message, and `OP_INTERNALKEY` pushes the Taproot internal key. Together they give signatures that are rebindable: valid for any output under the same key, not for one outpoint. The construction they are meant for is LN-Symmetry [9, 14], in which any later channel state can spend any earlier one, so that there is no revoked state to punish and nothing asymmetric for each party to hold. Two parties get a simpler channel from that. The change that matters here is to the number of parties. A penalty needs someone to blame and a state that one party holds against one

other, and extends poorly past two; a symmetric state signed by all members extends to any number, and the proposal's own motivation is that this "makes advanced constructs like multiparty channels practical." We take that object as given and ask what the bound becomes.

5.1 From edges to hyperedges

Let the channel set E of Section 3 be a set of hyperedges. Channel i has a member set $\mathcal{M}_i \subseteq N$ with $|\mathcal{M}_i| \geq 2$, a capacity χ_i, and balances $\ell_i(j) \geq 0$ for $j \in \mathcal{M}_i$ with $\sum_{j \in \mathcal{M}_i} \ell_i(j) = \chi_i$. A member spends only its own balance: a transfer of a from u to w inside channel i is feasible when $\ell_i(u) \geq a$, and is a state update that every member of $\mathcal{M}_i$ signs. The two-party channel is the case $|\mathcal{M}_i| = 2$. For $A \subseteq N$ let ∂A be the channels with members on both sides of the cut, let $S_A(n) = \sum_{i \in \partial A} \sum_{j \in \mathcal{M}_i \cap A} \ell_i(j)$ and $S'_A(n) = \sum_{i \in \partial A} \chi_i - S_A(n)$, and let O_A, I_A, K_A, K'_A be as before.

Lemma 2 (hyperedge cut bound)**.** *On the hypergraph the inequalities of Lemma 1 hold for every $A \subseteq N$ and every n, under every routing policy.*

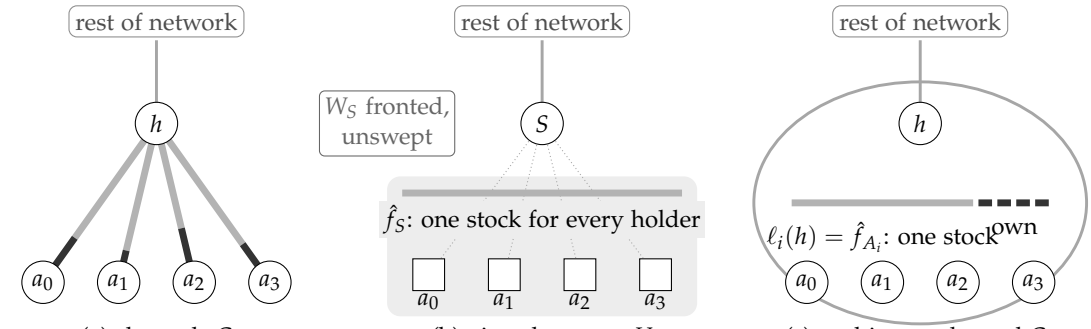
Proof. A transfer inside a channel $i \in \partial A$ either moves its amount between a member in A and a member outside, changing S_A and S'_A by that amount in opposite directions, or moves it between two members on the same side, changing neither. Channels outside ∂A are not in the sums. So $S_A(n) = S_A(0) - O_A(n) + I_A(n) + K_A(n) \geq 0$ as before, and likewise for S'_A; nothing in the argument depends on how attempts are chosen. □

The soft fork does not relax the bound. Conservation and nonnegativity belong to any object that holds value as balances, and Proposition 1 carries over word for word. What changes is how finely capacity is partitioned, which is what the provisioning problem of Section 3 was about. Figure 1(b) draws the same cut through a hyperedge.

Proposition 4 (pooling inside a hyperedge)**.** *Let channel i have members $\mathcal{M}_i = \{h\} \cup A_i$: a hub h through which the agents A_i reach the rest of the network, with $|A_i| = k$. Then (i) the inbound stock toward the agents on i is the single number $\ell_i(h)$, and net receipts into A_i over i are bounded by $\ell_i(h)$ at time zero plus what capacity events add, with no separate constraint on any one agent's receipts; (ii) a payment between two agents of A_i consumes none of $\ell_i(h)$; and (iii) the hub's provisioning problem on i is one newsvendor problem for the peak of the total its k agents hold, $\hat{f}_{A_i}$, in place of k problems for the $\hat{f}_a$.*

Proof. (i) is Lemma 2 on the cut A_i, where $S'_{A_i} = \ell_i(h)$ on channel i. For a single agent a the inbound on i is $\chi_i - \ell_i(a) = \ell_i(h) + \sum_{b \in A_i \setminus a} \ell_i(b) \geq \ell_i(h)$, so a receipt through the hub that the cut A_i admits, the cut $\{a\}$ admits. (ii) is the second case in the lemma's proof. (iii) follows from (i): the stock that has to be placed before demand is known is $\ell_i(h)$, and what exhausts it is the total held by A_i. □

The two-party channel is $k = 1$, and there the division of χ_i between the hub's part and the agent's part is the direction partition of Section 3. With k agents the division has $k + 1$ parts, but only the hub's is a stock that someone had to place by forecast; the agents' parts are their own money. Direction, which was a property of an edge, becomes a matter of who holds what inside a pool, and the pool reassigns it by signature. That is what Section 4 said of the virtual output, and here it is said without a server. Figure 4 puts the three objects side by side. The LayerTree leaf is a third form of the same statement: no direction at the member tier, since a leaf is paid to anyone by voucher, one output for all,



(a) channels C
k stocks, one per edge: $\sum_a \hat{f}_a$

(b) virtual outputs V
one stock, $\hat{f}_S$, plus W_S

(c) multiparty channel C_k
one stock, $\hat{f}_{A_i}$, nothing fronted

Figure 4: Where the forecast lives, for the same k agents and a hub. Light segments are stock placed by forecast before demand is known; dark segments are the agents' own balances. (a) Per-agent channels: the hub holds a separate stock on every edge, and the total is a sum of k forecast peaks. (b) An Ark: outputs have no direction at the holder tier, the server co-signs every transfer, its stock is forecast once for all holders, and value fronted at spends and refreshes waits unswept as W_S (Proposition 3). (c) A multiparty channel: one shared output whose hub part is the single stock for the group, forecast once, with nothing fronted and every update signed by all $k+1$ members (Proposition 4). The channel from h or S to the rest of the network carries a tier common to the three objects and is drawn without a stock.

and an operator needed to graft and to co-sign an update but not to pay, to redeem or to exit.

With N agents in N/k hyperedges the hub carries $\sum_i \hat{f}_{A_i}$, and Remark 1 applies inside each group with k in place of N: a factor of order $\sqrt{k}$ on signed imbalance, a factor that rises from 1 toward $\pi/2$ with k for agents that start from nothing, and nothing under shared drift. A population on per-agent channels is the multi-location newsvendor problem, and that serving several locations from one stock lowers the stock required is the result of Eppen [20]; the hyperedge is the instrument that performs the centralization, at whatever scale k its members can sustain.

5.2 The clock on capacity

Section 3 gave every capacity event a confirmation clock, and that clock is what made K'_A a forecast: inbound placed toward one agent could be moved toward another only by closing, splicing, or leasing on chain. A hyperedge has a second kind of capacity event. In the factory construction [10] the shared output funds a set of two-party subchannels, payments run in the subchannels at the speed and concurrency of ordinary channels, and the members re-divide the shared output among the subchannels by signing a new allocation, with no transaction on chain. Write $R_a(n)$ for the net capacity moved toward agent a by such reallocations inside a factory with member set $\mathcal{M}$. The bound on a's net receipts becomes

$$I_a(n) - O_a(n) \;\le\; S'_a(0) + K'_a(n) + R_a(n), \qquad \sum_{a \in \mathcal{M}} R_a(n) = 0, \tag{5}$$

where K'_a still carries a confirmation clock and R_a carries the latency of a signing round among $\mathcal{M}$. A per-agent forecast that turns out wrong is corrected from another agent's idle capacity in the time it takes $|\mathcal{M}|$ parties to sign, and only the factory's total remains

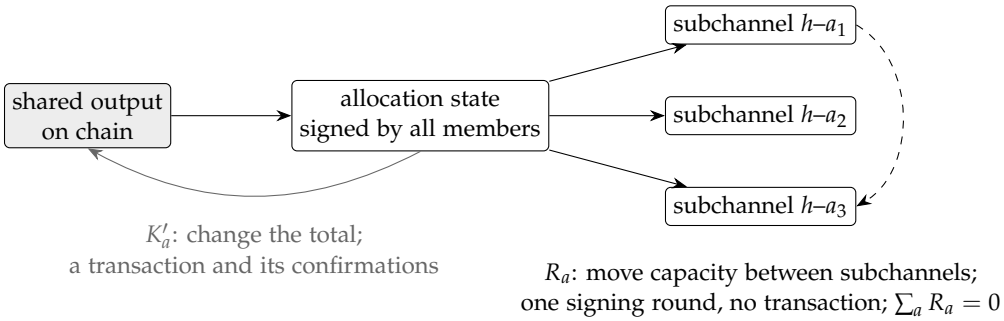


Figure 5: A factory's two clocks. The shared output is a chain object; the allocation that divides it among the subchannels is an off-chain state that every member signs. Moving capacity between subchannels, R_a in Equation (5), takes one signing round and conserves the total; changing the total, K'_a, takes a transaction on chain and its confirmations. Inside the factory a forecast made per agent can be corrected from another agent's idle capacity at the speed of a signature, and only the factory's total remains a chain-clocked forecast.

a forecast with a chain clock. A factory that reallocates every round has the bound of the single multiparty channel of Proposition 4, and one that never reallocates has the bound of k separate channels. The demand-shift row of Table 2, where a placement that had been right became worthless, describes capital that could not be moved; inside a factory it can be, among the members and no further. Figure 5 draws the two clocks. LayerTree's cohort rewrite is the factory's reallocation with a transaction for a clock, 151 vB per changed leaf in a cohort of 64 with only the changed owners and the operator signing, so a per-leaf forecast is corrected there on chain at that price rather than off chain for a signature.

5.3 What the pooling costs

The hyperedge buys its pooling with interactivity. Every update of a multiparty channel, and every reallocation of a factory, needs a signature from every member, so the object's liveness obligation is the conjunction of its members' and its counterparty set Γ is every co-member: not for custody, since no state exists that its holder did not sign, but for availability, since one member that stops signing freezes the pool at its last state and leaves the others to exit on chain. An exit publishes the latest state and waits out a relative timelock before it settles, and the state divides the output among all $k+1$ members, so one member's failure costs chain space of order k unless the state is itself a tree of templates, in which case an exit reveals a path and the pool survives only in the subtrees beside it. A covenant tree of the LayerTree kind is the other case: one member leaves in one transaction of $O(D)$ bytes and the output survives, recreated under the new root, for the rest. If each member independently fails or defects within the horizon with probability p, the pool survives it with probability $(1-p)^{k+1}$, which holds k to the order of $1/p$, while by Remark 1 most of what a pool of agents with nothing to spend down can gain has arrived by a k of a few tens.

The probability p is not only a matter of faults. The population this paper began with changes membership faster than a bilateral relationship can be amortized, and a hyperedge is exposed to that turnover in a way neither alternative is. Changing the membership of a multiparty channel respends its funding output, which takes a transaction on chain and every member's signature. A member that leaves cooperatively signs its own exit and costs the group one transaction, no more than closing a channel would have. A member that

simply stops forces the others to publish the state, at a chain cost of order k that all of them bear. Let each member depart at rate λ and let a fraction q of departures be uncooperative. The pool is forced on chain at rate $(k+1)\lambda q$, so that $p \approx \lambda q T$ over a horizon T and the pool's expected life is $1/((k+1)\lambda q)$. The chain cost of uncooperative departure, per agent and per unit time, is then of order $k\lambda q$ outputs. For per-agent channels it is of order λq, since a departing agent's close is its own; for an Ark it is nothing, since an absent holder's output expires and is swept by a server that was sweeping anyway; for a LayerTree it is the evictor's fee and no capital, since the operator grafts and evicts alone and an absent member's leaf is paid out on chain after its lease, 158 to 203 vB in a cohort sweep. A hyperedge makes every member pay for every other member's defection; an Ark, and a LayerTree, isolate their holders from one another. Because the pooling gain is nearly complete by a group of twenty and this cost grows in proportion to k, it is turnover and not capital that sets the group size; a population that churns quickly and leaves without notice belongs in an Ark, with hyperedges for whatever stable clusters it contains.

Section 4 observed that liveness distinguishes agents from humans and not Ark from channels. It does so more sharply here, because the hyperedge asks every member to be reachable for every other member's reallocation, which a population of always-online processes can supply and a population of phones cannot. The same proposal lowers the liveness price of the alternative: Ark variants built on these operations run rounds with no synchronous participation and let a server refresh an absent holder's output indefinitely [45]. That removes the obligation Λ of the V row for humans and changes little for agents, who met it already; and neither variant is described as changing what the server fronts or for how long, so W_S and the recovery question of Section 4 stand as they were. BIP 448 therefore narrows the advantage agents had over humans under Ark and opens a different one under the hyperedge.

5.4 The ratio with three objects

Write $\mathcal{D}_k$ for the liquidity duration of hyperedges of k agents and a hub, so that $\mathcal{D}_1 = \mathcal{D}_C$. Its locked capital is $\sum_i \hat{f}_{A_i}$, held for the horizon like the channel's, with no term in volume because nothing is fronted. One accounting decision has to be stated before the three objects are compared. An operator that serves agents holds a stock of its own toward them, which is $\sum_a \hat{f}_a$ on per-agent channels, $\sum_i \hat{f}_{A_i}$ in hyperedges, issuance capital of order $\hat{f}_S$ in an Ark, and the credit leaves and fronted grafts $\sum_a P_a + G$ of a LayerTree operator; and it needs inbound toward itself on its channels to the rest of the network, which its counterparties provision and which is the same peak of a sum whichever object it uses. Table 4 charges the channel the first and the server one $\hat{f}_S$. The hyperedge is charged the same way, its own stock and no more, so that each object carries one tier; the tier common to all three would move every ratio toward one and change no order. Then

$$\frac{\mathcal{D}_C}{\mathcal{D}_k} \approx \frac{\sum_a \hat{f}_a}{\sum_i \hat{f}_{A_i}}, \qquad \frac{\mathcal{D}_V}{\mathcal{D}_k} \approx \frac{\mathrm{Vol} \cdot \bar{L}/H + \hat{f}_S}{\sum_i \hat{f}_{A_i}}. \tag{6}$$

The first is the pooling gain at group size k and nothing else. In the second the ratio of stocks is at most one, with equality when one hyperedge holds everyone, and the forfeit term is the hyperedge's whole advantage over the server: large under the expiry reading of Section 4 and small under the round reading.

	$\mathcal{D}_C/\mathcal{D}_k$, oracle			$\mathcal{D}_C/\mathcal{D}_k$, trailing			$\mathcal{D}_V/\mathcal{D}_N$	
Demand cell	$k=5$	$k=20$	$k=N$	$k=5$	$k=20$	$k=N$	expiry	round
200 agents, bursty, drift 0, $H=8\tau$	1.30	1.44	1.50	0.99	1.01	1.03	1.77	1.03
200 agents, bursty, drift 0, $H=1\tau$	1.30	1.47	1.50	1.30	1.47	1.50	2.40	1.07
200 agents, bursty, drift $+60$, $H=8\tau$	1.01	1.01	1.01	1.00	1.01	1.02	1.04	1.02
200 agents, bursty, drift -30, $H=8\tau$	1.65	2.23	2.76	1.21	1.38	1.55	4.38	1.09
3 agents, steady, drift 0, $H=8\tau$	–	–	1.05	–	–	1.68	198.14	1.48

Table 7: Multiparty channels of k agents and a hub on the demand of Table 4, with the same margin and forecast rules. $\mathcal{D}_C/\mathcal{D}_k$ above one favors the hyperedge over per-agent channels. $\mathcal{D}_V/\mathcal{D}_N$ is at $k=N$ under the oracle forecast, for each recovery reading; above one favors the hyperedge over the virtual output. The few-agent cell has only $k=N=3$. Generated by `model/grid.ts`.

		Hyperedge, trailing forecast				VTXO
Demand cell	events	$k=1$	$k=5$	$k=20$	$k=N$	
200 agents, bursty, drift 0, $H=8\tau$	48 623	6 927	4 145	3 402	3 132	2 766
200 agents, bursty, drift 0, $H=1\tau$	6 092	944	907	907	907	907
200 agents, bursty, drift $+60$, $H=8\tau$	32 337	6 272	5 073	4 922	4 828	165
200 agents, bursty, drift -30, $H=8\tau$	56 671	12 931	9 884	9 040	8 856	8 856
3 agents, steady, drift 0, $H=8\tau$	32 256	12 703	–	–	1 287	4

Table 8: Failed events under the trailing forecast as the group grows, out of the events each cell attempts. $k=1$ is the per-agent channel of Table 6, and the VTXO column is the spends from an empty balance that every object fails. Generated by `model/grid.ts`.

5.5 Reference-model results

The reference model groups the agents of each cell k to a hyperedge and replays the demand of Table 4 against it. A receipt succeeds when the hub's balance in the group's channel covers it and a spend when the agent's own balance does, and the hub's stock is set by the same two forecast rules, applied to the total the group holds. At $k=1$ the computation is the channel model's, operation for operation, which the generator checks before it prints. Table 7 gives the ratios of Equation (6) and Table 8 the failed events under the trailing rule.

Under the oracle forecast the hyperedge is favored over per-agent channels wherever pooling has something to pool, and most of the gain arrives early. At zero drift the ratio is 1.30 at $k=5$, 1.44 to 1.47 at $k=20$, and 1.50 at $k=200$, against a limit of $\pi/2$; under a draining drift it is 1.65, 2.23, and 2.76; under shared accumulation it is 1.01 at every size. A group of twenty obtains about nine tenths of the zero-drift gain that a single pool of two hundred does, which is the practical content of the reflected case of Remark 1: for agents that start from nothing the pool that matters is small, small enough that the survival bound above leaves room for it when members rarely fail. Under the trailing forecast the capital ratio at zero drift over eight lifetimes is about one, 0.99 to 1.03, and the gain appears in Table 8 instead. The hub of a larger group forecasts a smoother total, and failed events fall from 6 927 on per-agent channels to 4 145, 3 402, and 3 132, against a floor of 2 766 that no object avoids. Figure 6 plots both against group size, with the $\pi/2$ limit and the $\sqrt{k}$ that signed imbalance would give. The few-steady cell, the channel's best case in Section 4, gains 1.05 under the oracle forecast; its 1.68 under the trailing rule comes with failures

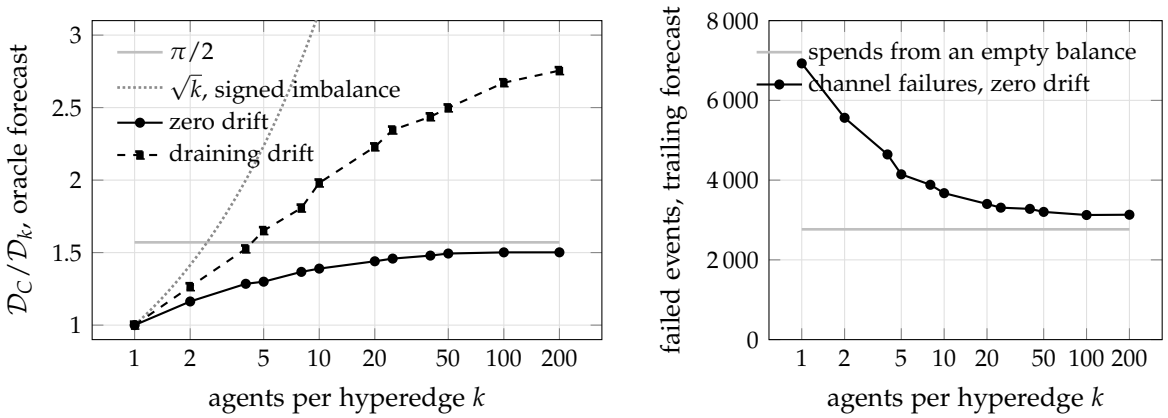


Figure 6: Left: the pooling gain of a multiparty channel over per-agent channels against group size, under the oracle forecast, on the zero-drift and draining cells of Table 7. At zero drift the gain rises toward $\pi/2$ and not beyond, because the agents start from nothing and their balances do not net (Remark 1); the $\sqrt{k}$ of signed imbalance is drawn for contrast and leaves the plot by $k = 10$. Right: on the zero-drift cell under the trailing forecast, failed events fall with group size toward the floor of spends from an empty balance, which every object fails alike. Generated by `model/grid.ts`.

falling from 12 703 to 1 287 and carries the caution that row was given before.

Against the virtual output the hyperedge at $k = N$ holds the stock the server holds and none of the forfeit lock. Under the round reading that is a ratio of 1.02 to 1.09 on the many-agent cells: the two objects cost the same capital, and the choice between them is a choice between a server in Γ with an expiry and every co-member in Γ without one. Under the expiry reading it is 1.77 to 4.38 where pooling has something to pool and 1.04 on shared drift, and on few-steady it is the forfeit lock of a server set against flows that a channel recycles within a round. The reading under which Section 4 found the virtual output not favored over the channel is the reading under which the hyperedge is favored over both. Once the proposal is assumed, then, the pooling claim of this paper no longer turns on the recovery rule: the rule decides between the hyperedge and the virtual output, and the hyperedge is ahead of the per-agent channel by the factor of Remark 1 either way. What the model does not credit to the hyperedge is anything for payments between members of one group, which cross no cut under it and draw on the recipient's inbound under per-agent channels. The demand of Table 4 has none, so these ratios understate the hyperedge for a population that trades with itself.

5.6 Keys alone, rebinding, and covenants

Three mechanisms put several parties behind one output, and this paper has used two of them. They are worth setting side by side on the terms the bound prices, because they differ in what must be signed and by whom, which is the liveness column of Table 3 and the turnover cost of Section 5, and in what an observer can tell, which is what Section 7 prices.

Keys alone. MuSig2 [35, 36] makes n keys one key: the output is a plain BIP340 key, the signature a plain signature, and nothing in consensus changes. Its nested form, proposed as a draft BIP [21], lets any participant of an aggregation be itself an aggregate, so that a

	keys alone: MuSig2, nested	rebinding: BIP 448	covenant tree: LayerTree
needs today	nothing	a soft fork	a larger soft fork, four tiers
an update is signed by	all	all	owner and operator; a voucher by its issuer alone
an exit is	the pre-signed tree, $O(\log N)$ transactions	latest state and settlement, $O(k)$ bytes	one transaction, $O(D)$ bytes, owner alone
a stale state is	penalised, or waited out	replaced	absent
stock placed by forecast	one per shared output	one per hyperedge	one per credit leaf, per member
who sees the structure	nobody, from the key; nesting hides a group inside a member	every member, from the state	the operator, and every member following the output

Table 9: Three ways to put several parties behind one output, on the terms this paper prices. Keys alone is MuSig2 [36] and its nested draft [21]; the covenant tree is LayerTree [39], measured on its own transaction model.

tree of signing groups produces one signature under one root key, and neither the verifier nor a co-signer outside a subgroup learns whether a key is one signer or a group. Both are n-of-n at every node; nesting adds leaf signers to a signature and never removes one. What a key alone does not give is a state. A shared output held by a key is spent by whatever its holders have signed, so a multiparty position built this way is a set of pre-signed transactions: the factory of Section 5 without symmetry, in which every state change is signed by all, a stale state has to be guarded by a penalty or a window, and a member leaves by the pre-signed tree, $O(\log N)$ transactions. The capital bound is the hyperedge's, since the stock is one per shared output whatever its key is made of; the liveness obligation is the hyperedge's, with every leaf signer the nesting adds.

Rebinding. BIP 448 [48] keeps the keys and changes what a signature binds to: the latest state spends any earlier one. The multiparty channel of Section 5 is the result, still n-of-n at each update and $O(k)$ bytes on chain when one member leaves without the others, with the pooling, the clocks and the turnover cost that section gives.

A covenant tree. A third mechanism takes the co-signers out of the exit altogether. LayerTree [39], a draft dated October 2026, commits a Merkle-sum tree of balances to one output and lets a member leave in one transaction of $O(D)$ bytes, about 1,020 vB at depth 16 on the draft's own transaction model, on its signature alone, with an operator that grafts members and co-signs updates but cannot delay an exit; the price is a larger set of opcodes than BIP 448's, in four tiers none of which is active. Table 3 carries it as T, Section 4 recomputes the bound under its lease, graft and credit-leaf terms, and Section 4.7 gives the throughput constraint it brings.

What nesting changes for this paper. Nothing in the bound and something in the observer model. The stock is per object and does not care how the object's key is built. But a member's key may be a group: a principal can be a required co-signer inside every agent's key, in a channel, a hyperedge or a leaf, with no script and with no co-member the wiser, which is the lever of Section 7.5 at the key level; and for the same reason a swarm can

be one controller behind many apparently distinct members, and nothing on chain or in the state says so. We reviewed the nested draft and posted findings on its pull request (October 9, 2026): the nonce-binding hash input was defined three ways, the theorems it cites are stated for a depth of at most two where the draft allows any, and its BIP 327 vectors were regenerated for sorted key lists. Its author has since answered the first (October 15, 2026); the rest we have not re-checked. Nothing here depends on the draft beyond its premise, that a nested aggregate verifies as a plain key.

The bound by tier. Table 9 orders the mechanisms by lineage. Ordered by what each asks of script instead, the objects of Table 3 form a ladder, and the bound is piecewise on it: at a rung an operator has the objects that rung admits and places by forecast the least stock among them, with the terms Sections 4 and 5 derived for each. Table 10 gives the rungs. Three things follow from reading the objects this way. The forfeit lock of equation (1) is not a property of pooled settlement but of pooling in an object that is re-funded on chain each round: a template hash alone moves the Ark exit tree from the server's signature to a covenant and leaves the round, its expiry and the lock where they were, as the draft's own comparison lists them [39, Table 12], and the lock leaves the bound at the first rung that admits an object whose balances change without the output being re-funded, which is BIP 448's for a hyperedge and, for a tree, the rung at which the tree becomes a payment object. The lowest LayerTree rung is not yet one in this paper's sense: MIN compiles the exit and the split, the one member-to-member transfer it supports, and both are chain transactions, so nothing moves off chain and no forecast term arises; channel leaves begin at MID or CCV, and vouchers, and with them the credit leaf of Section 4, at MID, or at CCV with CSFS; both tiers assume `OP_TX` in a tapleaf version its draft does not define, and FULL adds the smaller exit and the payout that pays its own fee. And the succession the ladder suggests, from the Ark to the tree as script permits, has been stated as intent by an implementer of Ark: were the opcodes LayerTree needs to activate, the company would build it for production [44]. We cite that as one implementer's statement and not as a design finding. Which opcodes it needs is, by the draft's own count, a different set at each rung, and the rung that needs least gives up the vouchers and channel leaves that make the tree a payment object.

6 Proof economics: a reward layer without identities

Whoever provisions the capital that Lemma 1 makes scarce needs to be paid for it, and a population without identities needs a way to pay that does not depend on knowing who is who. This section gives the mechanism and then the cost of running it on a chain that verifies no proofs itself.

6.1 Why weights have to be authenticated

Lemma 3 (identity splitting)**.** *Let a fixed reward pool P be allocated to claimants in proportion to integer weights w_u attached to scarce resources u. If each w_u is the value of a distinct settled object that a claimant controls, then splitting one claimant into m claimants whose weights sum to w_u leaves the coalition's payout invariant up to rounding. If weights are self-reported, a claimant with one object of value w can report it under m identities and receive m shares.*

The lemma is obvious and is stated because the second half is what an implementation does by default. A registry that records weights without authenticating that distinct claims

script has	objects	stock the forecast places	what the rung changes
BIP 342 as deployed	U, C, V, E	per edge, $\sum_a \hat{f}_a$; or the server's $\hat{f}_S$ plus the forfeit lock $\mathrm{Vol} \cdot \bar{L}/H$	the baseline: Lemma 1 and equation (1)
+ `OP_TEMPLATEHASH` (BIP 446)	the same four	the same	the Ark exit tree is a covenant and not the server's signature; the round and its expiry stay
+ BIP 448 (446, 348, 349)	adds C_k	$\sum_i \hat{f}_{A_i}$, pooled in each group, no lock; the pool forced on chain at rate $(k+1)\lambda q$	the server and the forfeit lock leave the bound; every member signs every update; its CSFS is MID's as defined, not the `0xC2` form FULL and CCV assume
+ `OP_CAT` (347) and `OP_TWEAKADD` (449), current leaf version	adds T at MIN	none: no transfer off chain, no graft	a shared output with a unilateral exit, 1,473 vB; no vouchers, no channel leaves
+ `OP_TX` and CSFS in `0xC0` (MID), or `OP_CCV` (443) with `OP_TX`	adds T with channel leaves, and vouchers (under CCV only with CSFS)	the operator's credit leaves and grafts, $\sum_a P_a + G$, and the byte-rate b_T; without vouchers no credit leaf, grafts alone	equation (3) and the constraint (4); 1,355 or 908 vB an exit
+ splice, bit and EC opcodes in `0xC2` (FULL)	the same	the same	1,020 vB an exit, 921 when the payout pays its fee

Table 10: The objects by what script has, and the stock a forecast places at each rung. The rungs are the drafts' as of October 2026 and form a ladder with one fork in it: BIP 448 and the `OP_CAT` tiers are independent proposals, and BIP 448's CSFS is what MID uses where BIP 348 defines it and what FULL, and CCV for vouchers, assume in `0xC2`; nothing else a tier needs is in BIP 448 (Section 4.1). Exit sizes are on the LayerTree draft's transaction model at $D = 16$.

are backed by distinct objects allocates to identities, and identities are free [15]. Authentication has two routes: the allocator learns who owns what, which defeats the purpose of a permissionless service market; or a claimant proves it controls a settled object of at least the claimed value and that no other claim uses the same object, without revealing which object. The second is the construction below.

6.2 The unique-exposure relation

Let $\mathcal{S}_n$ be the set of settled objects at step n in classes U, C, V, E, each with an identifier id_ρ, a value v_ρ, and an authority $\mathcal{A}_\rho$ that a key k_ρ satisfies. Let cm be a commitment to a claimant secret s, used only to link a claim to the identity that will be paid. Define

$$\mathcal{R}_{\mathrm{cap}} = \Big\{ \big((\mathrm{cm}, v, \mathrm{nf}, \mathrm{root}_n),\ (s, k_\rho, \rho, \pi_\rho)\big) : \mathrm{cm} = \mathrm{Com}(s),\ \rho \in \mathcal{S}_n \text{ via } \pi_\rho,\ k_\rho \text{ satisfies } \mathcal{A}_\rho, \\ v_\rho \geq v,\ \mathrm{nf} = \mathrm{PRF}_{k_\rho}(\mathrm{id}_\rho) \Big\},$$

where root_n is a Merkle commitment to $\mathcal{S}_n$ as the verifier knows it, π_ρ a membership path, and nf a nullifier keyed by the object's own authority rather than by the claimant. A claim is a statement $(\mathrm{cm}, v, \mathrm{nf}, \mathrm{root}_n)$ with a proof; the registry accepts it if the proof verifies and nf is not already in its nullifier set, and records $(\mathrm{cm}, v, \mathrm{nf})$ without learning ρ. The choice

of key matters: had nf been keyed by s, one controller could claim ρ once per identity by choosing a fresh s each time.

Proposition 5 (Sybil-bounded weight)**.** *If the proof system for $\mathcal{R}_{\text{cap}}$ is knowledge-sound and* PRF *is pseudorandom, then for every settled object ρ the total weight the registry attributes to claims backed by ρ is at most v_ρ, whatever the number of claimant identities, except with negligible probability. If the proof system is zero-knowledge, the registry learns nothing about ρ beyond $v_\rho \geq v$ and the nullifier.*

Every valid claim on ρ carries $\text{nf} = \text{PRF}_{k_\rho}(\text{id}_\rho)$, a function of ρ alone once k_ρ is fixed by $\mathcal{A}_\rho$, so a second claim collides and is rejected; a claim with a different nullifier on ρ needs a different key satisfying $\mathcal{A}_\rho$, which for a single-signer object does not exist and for a threshold object is handled by deriving nf from the aggregate key. This is Zerocash's nullifier argument [4] transplanted from spend-once to claim-once.

6.3 What the transplant leaves open

A nullifier stops a second claim on the same identifier. It says nothing about a claim on a successor identifier, and spending is the one thing a nullifier is designed to permit. Spend ρ to a fresh ρ' under a key the same controller holds, by a self-transfer on-chain or in a round, and $\text{PRF}_{k_{\rho'}}(\text{id}_{\rho'})$ is fresh, so Proposition 5 is satisfied per object while one unit of capital is counted twice. Under Ark the case is structural rather than adversarial: every refresh rotates the identifier, at a time the holder chooses.

The registry therefore takes claims by epoch. A claim is accepted only against $\mathcal{S}$ as frozen at epoch start, so a successor created mid-epoch is not claimable until the next; at epoch close a claim is revoked if its object is no longer settled, unless the set's maintainer has attested a successor link, which for class V is the server's record that a refresh reissued ρ as ρ' under the same key, and which a transfer to another key never receives.

Proposition 6 (epoch bound)**.** *Under the epoch rule and the assumptions of Proposition 5, the weight attributed in one epoch to any lineage of objects under one authority is at most the value settled under that authority at epoch start, whatever the number of identifiers the lineage passes through within the epoch.*

The epoch length should not much exceed the refresh cadence, or an honest holder waits a lifetime to claim a boarded output; and the successor attestation is one more statement the verifier trusts the server for, added to the co-signing trust it already carries. The reference model witnesses both propositions on finite cases and reproduces the hole: a registry without the epoch rule accepts eight claims on one object spent to eight identifiers. Figure 7 draws the rule and the hole it closes.

6.4 The service relation

A reward layer for liquidity provision wants a connector to prove that it honored requests over a window without revealing its customers. Let a connector κ receive requests over $[t_0, t_1]$ from counterparties who each sign an attestation $a_j = \text{Sig}_j(\kappa, t_j, \text{outcome}_j)$, and define

$$\mathcal{R}_{\text{svc}} = \Big\{ \big((\kappa, t_0, t_1, m, k),\ \{a_j\}_{j \leq m}\big) : \text{each } a_j \text{ verifies under a key in the counterparty set,} \\ t_j \in [t_0, t_1],\ \#\{j : \text{outcome}_j = \text{honored}\} \geq k \Big\}.$$

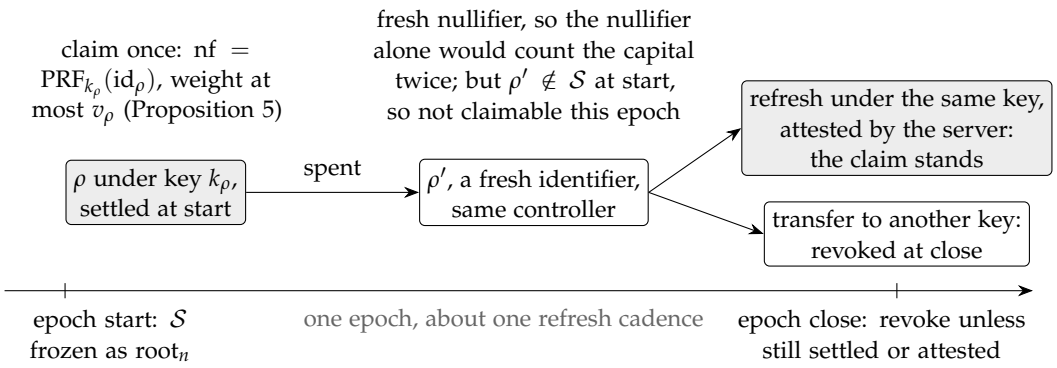


Figure 7: The epoch rule (Proposition 6). A nullifier keyed by the object's authority stops a second claim on the same identifier, but spending is what a nullifier permits, and a spend to a fresh identifier under the same controller carries a fresh nullifier. Claims are therefore accepted only against the settled set frozen at epoch start, so a successor made mid-epoch waits for the next; and at epoch close a claim whose object is no longer settled is revoked unless the set's maintainer attests a same-key successor, which an Ark refresh receives and a transfer to another key does not. Without the rule the reference model accepts eight claims on one object spent to eight identifiers.

The reward layer uses k/m as the service weight for the window; counterparties are not revealed; the keys must be in a set the verifier trusts, which reintroduces a registry of counterparties, one that attests only to key validity and not to transactions. Accountability attaches to the economic action rather than to a biography of the software, and that is the version of the sentence that can be verified.

6.5 Who verifies, and what it costs

Bitcoin consensus verifies no succinct argument. The verifier of a claim on $\mathcal{R}_{\mathrm{cap}}$ or $\mathcal{R}_{\mathrm{svc}}$ is therefore one of three parties: the reward allocator, a trusted server; the Ark server, already trusted for co-signing and well placed to hold root_n for class V; or an optimistic game in which an operator asserts the statement and any challenger can disprove it on-chain.

The costs separate by side. On the prover's side, $\mathcal{R}_{\mathrm{cap}}$ is a Merkle path and a PRF: with an arithmetic-friendly hash the circuit is on the order of 10^4 constraints, with SHA-256 on the order of 10^6, and either is a few seconds on commodity hardware under Groth16 [23]. $\mathcal{R}_{\mathrm{svc}}$ verifies m signatures in-circuit; a secp256k1 signature in non-native arithmetic costs on the order of 10^5 to 10^6 constraints, so the statement grows linearly in m and is the one whose memory budget sets a practical ceiling on window size. On the verifier's side, a trusted party pays a constant-time pairing check; the optimistic game pays what BitVM3 publishes [34]: an Assert of about 2.4 kvB, a Disprove of about 93 vB, an off-chain garbled circuit of about 41 GB for a Groth16 verifier that takes about six minutes to garble and six to evaluate, seven such circuits (288 GB) and some three and a half hours of setup when the garbling is checked by cut-and-choose, one honest challenger online throughout, and a committee honest at setup with one signer deleting its key afterward. Soundness is Theorem 7.2 of that paper under those assumptions, the random-oracle model, and a knowledge-sound proof system, which for Groth16 means a trusted setup per circuit (Section 9).

The economic consequence is a scale. The fixed cost of a permissionless verifier is the

garbled circuit, and it is paid per assertion: the setup garbles a circuit for each Assert, and a garbled circuit is good for one evaluation. It is amortized only across the claims that one assertion covers, so claims have to be batched into one proof per operator and epoch; the per-claim cost is then a share of that setup and of the Assert fee, plus a challenge period measured in blocks. A claim is therefore worth making only if its share of the pool exceeds its proving cost plus its share of the setup, which under Proposition 5 is proportional to the capital behind it, so proof cost sets a minimum capital to participate: roughly the proving cost times the ratio of total registered weight to the pool. That is the scale of a bond on a connector whose horizon is days and whose value is large against fees. It is not the scale of a payment, and a sentence of the form "Lightning needs BitVM-secured covenants" is true of the bond layer and false of the channel layer.

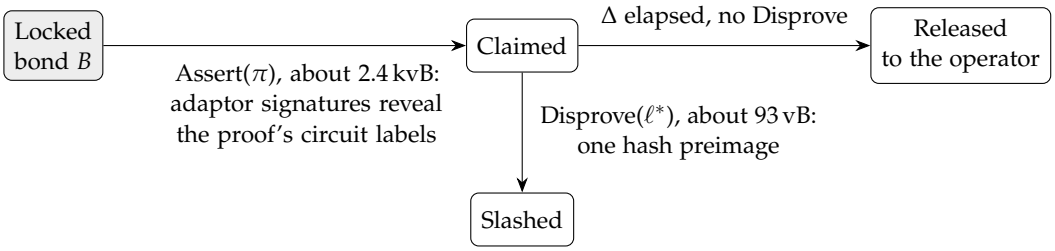
A settlement object can set a floor of its own beneath the proof layer's, and the LayerTree leaf does. Only the FULL tier's fee-from-payout exit pays its fee from the tree's value, 921 vB at $D = 16$; every other unilateral path needs a confirmed on-chain coin of the member's own: the exact exit that MID, MIN, CCV and every channel leaf use (1,020 vB), a channel leaf's force close (about 1,681 vB), and the credit-leaf fallback of a claim followed by the exact exit (2,656 vB). At 2, 10 and 50 sat/vB those are 1,842, 9,210 and 46,050 sat for the first; 2,040, 10,200 and 51,000 for the second; 3,362, 16,810 and 84,050 for the third; and 5,312, 26,560 and 132,800 for the fourth, below which the fallback costs more than it recovers. The grid's agents hold balances of order 10^3 to 10^5 sat, so at 10 sat/vB a balance at the lower end cannot leave alone at a profit, and a member with no on-chain coin has exactly one unilateral path, the FULL exit on a plain leaf: it cannot force-close a channel leaf and cannot exercise the credit-leaf fallback, so the Lightning-receive flow is enforceable only by members who hold a coin outside the object, which the population this paper began with, agents paying from low balances with no bank, may not. T's exit cost is a function of the member's holdings outside the object, which no other row of Table 3 has.

6.6 The bond as an object

A bond that is a number in a record, released or slashed by arithmetic on the same record, admits a negative slash, an over-release, and a bond declared after its owner's wealth is exhausted; the reference model's audit found all three in an earlier implementation. Under the tuple of Table 3 a bond is an object of class E: an on-chain output whose script releases to the operator after Δ unless a valid disprove consumes a connector output first. Its states are those of Figure 8: Locked, Claimed on a valid Assert, then Released after Δ with no Disprove or Slashed on one, with the release condition a statement in $\mathcal{R}_{\text{svc}}$ with k/m above the bonded threshold.

Proposition 7 (enforceable bond)**.** *Under the BitVM3 assumptions, an escrow $E(B, \kappa, \Delta)$ whose release condition is a statement in a SNARK-provable relation is enforceable against an uncooperative operator: a false Assert is slashed by any single honest challenger at a cost independent of B, and a true Assert is released after Δ. The bond is an account in the sense required exactly when the registry records the escrow's outpoint and the journal its state, and release arithmetic is replaced by the object's transitions.*

A native covenant, if one were activated, would replace E by E': no committee, no challenger, $\Delta \approx 0$ for template-enforced spends, and the release condition restricted to what script can check, which excludes SNARK verification without further opcodes. The model keeps E' as a comparison column and depends on nothing in it.



Setup: an n-of-n committee presigns this graph and one member deletes its key; the operator garbles the proof verifier, about 41 GB, seven circuits under cut-and-choose. Challenge: any one honest challenger evaluates the circuit off chain in about six minutes and, if the proof is false, learns ℓ^*.

Figure 8: The bond as an object of class E (Proposition 7), with the figures BitVM3 publishes [34]. A true Assert is released after the challenge period; a false one is slashed by any single challenger at a cost that does not depend on the bond. The committee at setup and the per-assertion garbled circuit are the costs Section 6.5 calls bond-scale.

BIP 448 is the concrete candidate for E', and it moves the line less than one might expect. CSFS lets a script verify a signature on a message, so a release condition of the form "these named attesters signed this window's outcome" becomes checkable in script: $\mathcal{R}_{\text{svc}}$ with its counterparty keys public and m small enough to fit a script, or with one aggregating key, is enforceable with no proof, no game, and no committee, at the price of the privacy the relation was written to keep. $\mathcal{R}_{\text{cap}}$, which hides which object backs a claim, still needs a succinct verifier and so still needs the optimistic game, and of the three mechanisms BitVM3 puts on chain [34] the proposal changes one. The proof is committed by adaptor signatures whose publication reveals the garbled circuit's input labels. That is already a tenth the size of a hash-based commitment, and checking those signatures against a stack message instead of the transaction would not shrink them, so the Assert stays at its published size. The covenant BitVM3 emulates with its committee is a binding between inputs: a withdrawal may spend the reserve only together with a valid Assert. `OP_TEMPLATEHASH` commits to a transaction's outputs and by design not to the other coins it spends [47], so it cannot state that binding, and the committee stays in Γ of the E row. What CSFS does replace is the one-time hash-based signature that holds an operator to a single public input across transactions, since two signatures under one key on different messages are an equivocation that a script can check. That matters where the statement is supplied when the assertion is made and not where it is fixed at setup. We therefore leave the published BitVM3 figures as inputs, unadjusted.

The LayerTree voucher is not an escrow, and the difference says what an escrow is for. A voucher is a signature by a source leaf's owner over that leaf and a new leaf to be carved from it; anyone holding it redeems it alone with a claim, 1,618 vB at $D = 16$, which checks the signature with CSFS and nothing else. It is voidable by the issuer, who can advance the source's counter first, or by the source's evictor after the lease, until the redemption confirms; a cumulative voucher re-signed for the running total needs no contest window, since the latest dominates; the hash-locked form paired with an HTLC on the same hash is an ordered exchange and not an atomic swap, under five rules no script enforces; and a dedicated credit leaf removes the voiding, leaving the issuer's liveness as the holder's exposure. Against Proposition 7: the voucher enforces no statement about the world, slashes nothing, has no deadline because script cannot give it one, and its collateral is

Observer	C channel	V VTXO	E escrow
O_{chain}	opens, closes, splices, force-close scripts	round transactions, exits, offboards; aggregate per round	deposits, Asserts, Disproves, releases
O_{gossip}	`channel_update`, `node_announcement`, policy	nothing per holder	nothing
O_{peer}	HTLC amount, timing, CLTV, preimage on one hop	the HTLC the server forwards on its own channel; the holder is behind the server	nothing
O_{server}	none	every transfer: parties, amount, time, co-signed	operator's own escrow

Table 11: What each observer sees under each object.

the sum it pays. It needs no challenge period, no challenger, no committee and no proof system, because it claims nothing beyond "this leaf may be carved so", whose only risk is the issuer's own voiding or absence. It can carry the connector's payment, and the role of Ark's out-of-round transfer with a different trade, undone by the issuer alone until confirmed against undone by sender and server together; it cannot carry the connector's bond, and the release of $\mathcal{R}_{\text{svc}}$ on an attested service ratio is not a LayerTree predicate.

7 Agent swarms: observers, coordination, and what a coalition can earn

The objects of Sections 4 and 6 were introduced for a population of honest agents. The same objects are available to a population under common control that wants to appear independent, and the question this section asks is what that population can do with them and who would see it. Following Simmons [54], we fix the warden first.

7.1 Observers by object

There are four observers, and each object exposes a different trace to each (Table 11). Under the channel no single observer sees a whole path, which is the property the privacy literature studies and which probing and timing attacks partially erode [25, 27, 43]. Under the VTXO one observer sees everything within its Ark: the server co-signs every transfer and so knows sender, receiver, amount, and time for every payment it hosts. A server is a total warden for its holders. The proofs of Section 6 let it not publish what it sees, and a permissionless escrow lets it not be trusted to allocate rewards, but its view is reduced by neither. BIP 448 would change the O_{peer} row at the margin: its authors expect rebindable signatures to simplify the move to point time-locked contracts, under which two hops of one payment no longer share a hash to correlate on, while amounts and timing remain. Under T the operator co-signs grafts and updates and sees every transition of the output, as does anyone, but a voucher in flight and a payment in a channel leaf are not shown to it: O_{server} is partial where under V it is total, and O_{chain} sees one output's whole spend chain.

7.2 Three things called coordination

The word covers three activities that need different treatment. *Explicit messaging*: onion payloads carry TLV records and custom records are a specified use [29]; gossip carries aliases, colors, and policy fields any node may set [30]. A population that wants to exchange symbols can do so in these fields at near-unbounded capacity, and detection is traffic analysis, not content analysis; steganography through amount classes and timing is the expensive way to do what the protocol permits cheaply, and is not the mode to study first. *Implicit coordination*: agents infer shared state from public events with no message, a fee update, a failure pattern, an on-chain open. This is not a channel in the information-theoretic sense and has no capacity; it is a focal-point structure [49], and its monitoring signature is correlation of control loops rather than any payload. *Capital coordination*: agents under common control claim rewards, place capital, or manufacture demand as if independent. This is the mode the reward layer must resist.

7.3 What a coalition can earn

Let a coalition of agents share a controller and a covert channel of unbounded capacity in the explicit mode, and let the reward layer use the registry of Section 6 and the escrow as its bond.

Proposition 8 (coordination gain, conditional)**.** *Under Propositions 5 and 6, the coalition's authenticated weight is at most the value of the distinct settled lineages it controls. Under the placement result of Table 2, the gain from placing a fixed budget with full knowledge of persistent demand is the stationary-hotspot effect, six points over random, and nothing after a shift. Under a service weight that nets value circulated among a claimant's own attesters, manufactured demand earns no service weight. Then the coalition's gain from coordinating, in the tested model, is bounded above by the stationary-placement effect on its own capital and is zero under demand shift, whatever the capacity of its channel.*

The proposition is a conjunction of three conditions, and each is an attack surface. The first is the registry's proof system and the verifier's view of $\mathcal{S}_n$. The second is a demand process with more persistence than the campaigns tested. The third is a netting statistic that discriminates, and here the record is a warning: on the campaigns' coalition fixtures a network-wide gross-to-net churn ratio failed, with benign multilateral commerce netting more strongly than either labeled pattern, and the reference model reproduces the failure and adds one. Netting restricted to a claim's direct attesters catches a reciprocal pair, misses a four-member relay, and saturates toward one when benign trade is dense enough that most counterparties attest. What separates the fixtures is the share of a claimant's volume lying on a directed cycle through it, of bounded length, whose every edge recurs above a multiplicity threshold; and on those fixtures the relays are several hundred times louder than background, which is a sanity check and not a result. Whether the statistic survives benign structure in which short cycles through a busy node recur for ordinary reasons is open, and a coalition that controls its own rate can stay under any fixed threshold.

None of the three conditions is a covert-channel problem. All three are capital and observation problems, and that ordering is the section's main claim: a capacity-versus-detectability curve for a steganographic channel over payment shape [11] would be a real result and would bound a quantity that Proposition 8 already bounds by other means.

7.4 The warden's statistics

Formally the warden observes a trace from one row of Table 11 and decides between H_0, that the agents draw independent demand from a declared process, and H_1, that a subset shares a controller. The likelihood ratio is not computable in general because H_1 is composite. Three statistics are: repeated-cycle share through the claimant; nullifier collision rate at the registry, exactly zero under honest claims and positive under duplicated capital; and service-versus-demand inconsistency, a claimed k/m against a demand estimate that does not support it. Under V the server computes the first completely and needs no probing; under C no one does, and it must be estimated from partial hops, which the campaigns modeled as an observer seeing 11–13% of events. A monitor that wants the capital-coordination signal should therefore sit at the server and at the registry, not on the gossip network, and should measure whether cross-server circulation defeats it. Under T there is a third place to stand that needs no server's leave, the output's spend chain, on which the contention attack of Section 4.7 leaves a run of carves on one leaf at a rate near κ, visible to anyone; a statistic and not a defence, since the defence is the honest bid, β, and a smaller output.

7.5 What a swarm does with the objects

The analysis so far assumes the coalition uses the objects as a holder. Six further possibilities change the picture, and each is stated as a conjecture with the experiment that would test it.

The swarm runs the server. An Ark server needs on-chain capital for rounds, a channel set for ramps, and continuous liveness, and the last two are what a population of agents has in surplus. A swarm-run Ark is a private clearing house whose chain footprint is one round transaction per interval with an aggregate output set, and whose Lightning footprint is a single node's channel set; every intra-swarm transfer is out-of-round, co-signed by the swarm's own server, visible to no one else. Relative to N gossip-visible Lightning nodes this is a large reduction in observable surface at no cost the swarm does not already bear, and it inverts the recommendation of the previous subsection: the monitor cannot stand at a server the swarm operates. The warden's question becomes which servers are swarm-run, a chain-fingerprinting problem on round cadence, output-value distribution, and the ratio of out-of-round churn to on-chain exits, rather than a payment-graph problem.

Ramps remain. The server-tier term of Section 4 is also the swarm's exposure. Whatever enters the Ark over Lightning crosses the swarm server's channels, which are gossip-visible and whose inbound capacity came from a peer who provisioned it; whatever boards or exits is on-chain. A swarm can hide circulation; it cannot hide income arriving or capital leaving without a mixer, which is orthogonal to this stack. Ark reduces the observable surface from flows to net flows at the perimeter, and the detection budget should move accordingly.

Cross-server circulation. Two swarm-run Arks paying each other over Lightning look like two independent servers with ordinary bilateral settlement; each server's view is total inside and nil across, and a cycle through both is two intra-Ark legs no outsider sees plus two Lightning legs that look like server-to-server flow. The repeated-cycle statistic needs

an observer who sees the cycle, and with two servers no single row of Table 11 does. This is the fixture to build.

The swarm is the channel. Under BIP 448 a swarm needs no server to obtain what the swarm-run server gave it. A multiparty channel whose members are all the swarm's agents is funded by one output that a key-path spend makes indistinguishable from a single signer's, has no rounds and so no round cadence to fingerprint, appears in gossip as one channel or as none, and settles every transfer inside the swarm as a reassignment of balances that only members see. The perimeter argument survives, since value still enters and leaves over some member's channels to the rest of the network, and so does the capital bound of Proposition 8. What does not survive is the chain-fingerprinting route to the question of which servers are swarm-run, because there is no server. The object cuts the other way when membership is mixed: every member signs every state, so each honest member of a hyperedge is a total observer of it, and a monitor that wants the capital-coordination signal should want to be a member. Table 11 would carry an O_{member} row under C_k that reads like the O_{server} row under V, held by $k+1$ parties instead of one.

The registry's privacy is symmetric. $\mathcal{R}_{\text{cap}}$ proves capital without revealing which object, and the registry learns a value and a nullifier and nothing about control structure. That is what a swarm wants: proportional reward without disclosing that thirty claimants share a controller. Propositions 5 and 6 make the reward capital-proportional, which is the right property against a poor swarm pretending to be rich and no property against a rich one. The bound is on excess return, not on presence; a well-funded swarm receives exactly its capital share with no one able to say it is one entity, and whether that is a problem depends on what the reward layer is for.

Enforceable side-contracts. Cartels are unstable because side-payments and punishments among parties who cannot go to court are unenforceable [55]. An escrow whose release condition is a provable statement is an enforcement device for exactly such agreements, and the costs that make it unattractive for honest per-payment use, a circuit garbled for every assertion and a committee honest at setup, weigh little on a swarm: one operator, one assertion per contract period ("member x did not deviate from schedule s"), and a committee the swarm itself runs, so that the honest-at-setup assumption is satisfied trivially when setup and participants are one principal. A MuSig2 multisig with a swarm-run key-deletion ceremony gets most of the way without BitVM. The mechanism introduced to make connector bonds honest also makes swarm-internal commitment devices honest; the open question is whether rival swarms, or a swarm and a human counterparty, can use the same object, since that is where enforceability was previously missing [12].

Same weights, no message. Implicit coordination was described above as plausible. For a population of agents instantiated from the same model it is the base case: agents sharing weights share tie-breaking and converge on the same fee, the same retry schedule, the same placement, without a message. The proposed monitoring signature, correlation of control loops, fires on this constantly, because the false-positive class (same model, independent principals) and the true-positive class (same model, one principal) produce the same correlation. A warden that wants the common-control signal has to separate it from common training, and we conjecture that payment traces alone cannot. The test is cheap:

two populations on the simulator, one sharing a policy seed with independent budgets and one sharing a controller, and whether any of the three statistics distinguishes them.

The lever is the authority field. Every case above assumes the swarm holds its own keys. The tuple has the field where the safety lever sits: $\mathcal{A}$, the spend authority. A policy response is not a better statistic but an authority that includes a principal-held co-signer with time-bounded delegation, so that an agent spends within a budget and window without a co-signature and beyond it needs one. Lightning has no natural place for this; Ark does, because the server already co-signs every transfer, so a principal-run or regulated server is a policy enforcement point by construction, which is the same fact that makes a swarm-run server a policy evasion point. The observer table should carry an $O_{\text{principal}}$ row, since for alignment purposes the observer who matters most is the operator the agents nominally work for, and the failure of interest is a population that routes around its own principal's view. BIP 448 gives the lever a second seat and a weaker one. Its proposers describe CSFS as enabling delegation: a principal signs a message naming an agent's key and a script checks that signature before the agent's, so that delegated spend authority becomes expressible on any Taproot output and not only where a server co-signs. It falls short of the delegation asked for above in two respects. Script can delay a spend but cannot expire one, so a window closes only when the principal moves the coin; and the proposal introspects no amounts except through a template that fixes every output, so a budget is a set of templates the principal has signed. In a hyperedge the principal has a simpler option, which is to be a member. Key aggregation gives the lever a third seat, and one available now: with MuSig2 the agent's key in any of these objects can be a two-of-two of agent and principal, and under the nested draft a group's key can stand wherever one member's does, so the principal signs every spend with no script and without the co-members knowing (Section 5.6). The same fact serves a swarm, whose members' keys can share a signer that no observer can see. A LayerTree operator is a weaker enforcement point than an Ark server, and so a weaker evasion point for a swarm that runs one: by the draft's own table of powers it cannot delay an exit beyond [C], move or reduce another leaf, or freeze value it did not graft and fund; what it can refuse is a graft and its signature on an update.

8 Preregistration

The following is to be preregistered on the reference simulator extended with the class-V ledger and the registry; no live component is required for any of it. Stopping rules, multiplicity, and the practical-equivalence band are as in the earlier campaigns [18], and analysis code must canonicalize graph and candidate order.

1. Pin the recovery rule. Determine from the implementation, not the documentation, whether a server-held HTLC output from a Lightning spend is presented to the next round or swept at expiry. This fixes $\bar{L}$ and the sign of every cell in Table 4.
2. The liquidity-duration grid. Demand family with declared parameters: agent count; per-agent burstiness (imbalance envelope over own volume); per-agent drift at zero, positive, and negative values with the population's drift correlation stated; horizon relative to lifetime; forecast rule (oracle and trailing); and the share of inflow arriving over Lightning versus by boarding. Compute $\mathcal{D}_C$ and $\mathcal{D}_V$ with the server-tier term

charged and two-way volume, report the ratio with a cluster-robust interval over sampled topologies, and report failed events per forecast rule beside it, since the duration does not price them. The claim of Section 4 is falsified if the ratio does not exceed one on the uncorrelated many-agent cells under the pinned recovery rule, or does not fall below one on few-steady.

3. The hyperedge grid. On the same demand family, with group size k, the share of flow that is between members of one group, and whether agents start with balances as added parameters, compute $\mathcal{D}_k$ under both forecast rules and report $\mathcal{D}_C/\mathcal{D}_k$, $\mathcal{D}_V/\mathcal{D}_k$, and failed events. The claim of Section 5 is falsified if on zero-drift cells whose agents start from nothing $\mathcal{D}_C/\mathcal{D}_k$ under the oracle forecast exceeds $\pi/2$ by more than the equivalence band or fails to exceed one at $k \geq 5$; if it exceeds one by more than the band under shared drift; or if it does not grow as $\sqrt{k}$ when agents start with balances large against their imbalance. Add a per-member departure rate λ with an uncooperative fraction q; report the pool's realized lifetime against $1/((k+1)\lambda q)$ and its chain cost per agent against $k\lambda q$, with the same turnover applied to per-agent channels and to the Ark; and report, for each cell, the group size that minimizes capital and chain cost together.

4. The LayerTree cell. On the same demand family, with the receive share, the top-up period T_u, the lease period T_ℓ, the settle window W, the funding mode and κ and β as parameters, compute $\mathcal{D}_T$ with its credit-leaf and graft terms and the byte-rate b_T beside it, on the draft's transaction model with its omissions stated; report the ratio of Equation (3) against $\mathcal{D}_C$ and $\mathcal{D}_V$, and the contention cost of Section 4.7 against the honest bid. Price bytes at 2, 10 and 50 sat/vB and capital at 2 percent a year in bitcoin terms, a liquidity-lease rate, with the quarter of a percent of Bhatia's reference rate reported beside it, and report the receipt rate at which the byte-rate's annual cost equals the carry of the capital the leaf saves, at each fee rate and under both forecast rules. The claim is falsified if $\mathcal{D}_C/\mathcal{D}_T$ does not exceed one on receive-heavy cells at a top-up period shorter than the horizon. On the bytes the prediction is a break-even near 23,000 sat a round at 10 sat/vB and 2 percent against the oracle stock, about two bitcoin a year of receipts; the item is falsified if the computed value lies outside a factor of two of it. The leaf is not claimed to pay at the grid's scale, where the indicative figure says it does not.

5. Prover cost. Implement $\mathcal{R}_{\text{cap}}$ with a real proof system over the simulator's settled set (a Merkle path and a PRF) and report prover time and memory against a stated budget.

6. Identity splitting and churn. One capital source, 1 to 32 identities, all claims through $\mathcal{R}_{\text{cap}}$: aggregate reward invariant, every duplicate detected. Then one identity, one source, spent to a fresh identifier 1 to 32 times within an epoch, with and without an attested successor: aggregate reward invariant in the epoch, revocations at close equal to unattested spends; a registry without the epoch rule fails by construction and the failure is reported as such.

7. Warden rows. Run the common-control fixtures through O_{peer} under C, O_{server} under V, and a mixed relay set spanning two servers. Report ROC per row. The prediction is that O_{server} dominates O_{peer} and that the two-server set degrades both.

8. Same weights versus same controller. Two populations, one sharing a policy seed with independent budgets and one sharing a controller, through the three statistics. The prediction is that none separates them.

9. Enforceable collusion. Two agent populations setting spreads in a toy market, one with an escrow-enforced side-contract and one without; report the sustainable collusive margin in each.

10. Only then, as a bounded appendix, a codebook over amount and timing, a prior over payloads, and a guessing-advantage task for O_{peer}, reported as a capacity–detectability curve read against Proposition 8.

9 Limitations

Three trust assumptions are not discharged, and the first two run through everything. The Ark server is trusted not to censor and not to collude with a sender on a pending payment; this has the shape of the assumption the channel model makes about a counterparty, concentrated in one party per Ark, and Section 7.5 says what follows when the party is the adversary. The BitVM3 committee is trusted at setup and one signer is trusted to delete a key; that is weaker than a federation and stronger than consensus, and a native covenant that commits to sibling inputs would remove it and not the first; BIP 448 as drafted does not (Section 6.6).

The third is confined to the proof layer of Section 6 and is easy to pass over, because the theorem we cite takes it as given. BitVM3's soundness assumes a knowledge-sound proof system whose reference string is a public parameter [34]. For Groth16 that string comes from a setup run once per circuit, and whoever keeps the randomness of that setup can forge proofs for the circuit [23]. Propositions 5, 6, and 7 therefore hold only if the setup randomness for the circuits of $\mathcal{R}_{\text{cap}}$ and $\mathcal{R}_{\text{svc}}$ was destroyed, which a multi-party ceremony reduces to one participant discarding its share, as in Zcash [26]. The assumption has the committee's shape, one honest party at a ceremony, and is separate from it. A covenant removes neither it nor the server, and it applies equally when the verifier is a trusted party checking a pairing and no game is played. A deployment can confine it to a single circuit by proving its relations in a transparent system and verifying that proof inside one fixed Groth16 circuit, at a prover cost that the figures of Section 6.5 do not include.

Nothing is measured live. The simulator results of Table 2 are on synthetic hidden state and sampled topology; the grid of Table 4 is a model whose recovery rule is pinned by reading one implementation at one commit and not by measurement in production, and whose issuance-inventory parameters are operator configuration we do not have; the proof system is simulated and the registry's tests witness logic, not cryptography; the BitVM3 figures are an implementer's published numbers at a date and may be revised. The coalition bound is a conjunction of conditions, one of which (a discriminating statistic) the reference model shows failing in two forms, and the remaining statistic is demonstrated only on fixtures where the signal is hundreds of times louder than background. Channel balances are not a public settled set, so a claim on class C capital can prove existence and capacity from gossip but not directional balance; that is the public–private split the whole problem began with, and proofs do not cross it.

Section 5 is conditional on a soft fork that is a draft and may never activate, or may activate in another form. The multiparty channel it analyzes has no deployed implementation; its failure model is one independent probability per member, which a member that defects strategically does not obey; and the reference model gives it a hub, a population that never pays itself, and a membership that never changes, so the turnover cost of Section 5 is derived and not measured. The constant $\pi/2$ is a diffusion limit for agents that

start from nothing, and the model's agents all do, so the hyperedge tables do not exhibit the $\sqrt{k}$ regime of Remark 1; Table 1 exhibits it on the same demand law with the reflection removed, which changes the agents and not the object. Section 5.6 compares three mechanisms of which one is deployed, one is a draft soft fork and one a draft protocol on inactive opcodes; the nested signing draft is under review, with findings of ours partly answered; nothing in the bound depends on either draft. The ladder of Table 10 orders proposals whose tapleaf versions and opcode numbers are not settled, so its rungs are the drafts' as of October 2026 and not consensus's, and the statement of intent it cites is a post by one implementer, not a specification. The LayerTree figures are the draft's own, on a transaction model that omits witness length prefixes and, in its baselines, anchors, fee children and every round-level cost of a deployed Ark; its security argument is a UC proof in a full version we have not read; the 122 vB top-up and the 151 vB renewal are carried from its tables rather than reproduced from its measured scripts; and its throughput assumption is policy, with fee pinning outside it.

10 Conclusion

On Lightning an agent's edge is not in routing, because the bound on delivery is a conservation identity over the cut around it and no policy relaxes it. The bound is a property of the object: bilateral, pre-funded, direction-partitioned, so that the forecast it forces is made per edge and does not pool. A server-mediated object moves the forecast to the server, where it pools by a factor of order $\sqrt{N}$ on signed uncorrelated imbalance, by a constant near $\pi/2$ for agents that start from nothing, and not at all on shared drift, and whether that is worth the server's forfeit lock depends on a recovery rule; read off the implementation, the rule is expiry, and on the model's demand the pooled object is not favored. The multiparty channel that BIP 448 would make practical obtains the same pooling inside a hyperedge with no server and no forfeit lock, most of it at a group of twenty, at the price of every member's signature on every reallocation and of every member's exposure to every other's departure, which an Ark does not impose; the cut bound itself survives the change of graph. A covenant tree of the LayerTree kind removes the co-signers from the exit and the forfeit from the spend, places the operator's stock per member and pays for a shorter forecast in bytes, and makes throughput on one output a constraint of its own; and a key that is a group changes who can be seen, not what is bound. A claim-once registry makes a reward layer capital-proportional without identities, once an epoch rule covers the gap a nullifier leaves; an optimistic escrow makes a bond an object; and both cost what verification costs on a chain that does not verify, which is bond-scale and not payment-scale. For a population of agents under common control, what it can say to itself over the payment layer matters less than what it can prove and where it stands: the gain from coordination is bounded by authenticated capital under stationary demand, a swarm that runs its own server removes the observer the bound relies on, an enforceable escrow gives it the commitment device cartels lacked, and agents sharing weights coordinate without a message in a way traces cannot distinguish from independent use. The experiments that test each of these are listed and none has been run.

References

[1] Kenneth J. Arrow, Theodore Harris, and Jacob Marschak. Optimal inventory policy. *Econometrica*, 19(3), 1951.

[2] Moshe Babaioff, Shahar Dobzinski, Sigal Oren, and Aviv Zohar. On Bitcoin and red balloons. In *ACM Conference on Electronic Commerce (EC)*, 2012.

[3] bcSatoru. The price of being receivable: What inbound liquidity actually costs on Lightning, 2026. Spark research note, September 21, 2026, on mid-2026 market rates: liquidity ads 0.5 to 1.5 percent a year, Lightning Pool 1.5 to 3, Magma about 2.6 for channels of a bitcoin and more and over 4 under a million sat. Advertised rates, not executed trades. https://www.spark.money/research/lightning-inbound-liquidity-acquisition-cost, accessed October 16, 2026.

[4] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *IEEE Symposium on Security and Privacy*, 2014.

[5] Ferenc Béres, István András Seres, and András A. Benczúr. A cryptoeconomic traffic analysis of Bitcoin's Lightning Network. *Cryptoeconomic Systems*, 1(1), 2021.

[6] Nik Bhatia. The Lightning Network reference rate, 2019. Medium, March 11, 2019, part 4 of the series of that name. Defines the node accrual rate $[(p+f)/p]^{52{,}560/n} - 1$, routing fees f on average channel balance p over n blocks, annualised, and the reference rate as an aggregate of disclosed node rates; its examples are 0.25 and 0.18 percent. https://timevalueofbtc.medium.com/the-lightning-network-reference-rate-98e41a9dadfa, accessed October 16, 2026.

[7] Bitcoin Optech. Ark, . Topic page, https://bitcoinops.org/en/topics/ark/, accessed September 30, 2026.

[8] Bitcoin Optech. OP_CHECKTEMPLATEVERIFY, . Topic page, https://bitcoinops.org/en/topics/op_checktemplateverify/, accessed September 30, 2026.

[9] Bitcoin Optech. Eltoo, . Topic page, https://bitcoinops.org/en/topics/eltoo/, accessed October 5, 2026.

[10] Conrad Burchert, Christian Decker, and Roger Wattenhofer. Scalable funding of Bitcoin micropayment channel networks. *Royal Society Open Science*, 5(8), 2018.

[11] Christian Cachin. An information-theoretic model for steganography. In *Information Hiding (IH)*, 1998.

[12] Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110 (10), 2020.

[13] Lin William Cong, Xi Li, Ke Tang, and Yang Yang. Crypto wash trading. *Management Science*, 69(11), 2023.

[14] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. eltoo: A simple layer2 protocol for Bitcoin, 2018. Whitepaper.

[15] John R. Douceur. The Sybil attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.

[16] Patrick B. Dugan. bitcoin-liquidity-formal: Lean 4 proofs of the accounting core, 2026. Cut invariant and prefix cut bound, policy invariance, peak-of-pool bound, cross-class conservation, server headroom, and a counterexample to the cumulative-allocation reading of the server bound. Deterministic abstractions; the stochastic pooling scale and the liquidity-duration grid are outside its scope. https://github.com/patrickdugan/bitcoin-liquidity-formal, accessed October 23, 2026.

[17] Patrick B. Dugan. Liquidity on trial: Counterexamples and a resource-sensitive semantics for agent payment networks, 2026. Spiral adversarial follow-up, September 11, 2026. https://github.com/patrickdugan/Spiral/blob/main/paper/liquidity_on_trial.md.

[18] Patrick B. Dugan. Spiral: preregistered campaigns, audit, and reference model for agent liquidity on Lightning, 2026. Repository, https://github.com/patrickdugan/Spiral. Public-topology campaign at commit a0a3140; audit at commit 618a12e; reference model and table generator at commit fa7dab3.

[19] Patrick B. Dugan, Daniel P. Pizarro, and Lihki J. Rubio. Graph re-write using ghost nodes for target value rebalancing, 2021. U.S. Patent Application Publication US 2021/0004796 A1, published January 7, 2021; application 16/920,416 filed July 2, 2020.

[20] Gary D. Eppen. Effects of centralization on expected costs in a multi-location newsboy problem. *Management Science*, 25(5), 1979.

[21] Beulah Evanjalin. BIP draft: NestedMuSig2 for BIP340-compatible signatures, 2026. bitcoin/bips pull request 2297, opened September 21, 2026; head 4523ee0 of October 15, 2026. Our review of the September head is linked from the pull request. https://github.com/bitcoin/bips/pull/2297.

[22] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 1956.

[23] Jens Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT*, 2016.

[24] Paolo Guasoni, Gur Huberman, and Clara Shikhelman. Lightning network economics: Channels. *Management Science*, 2024.

[25] Jordi Herrera-Joancomartí, Guillermo Navarro-Arribas, Alejandro Ranchal-Pedrosa, Cristina Pérez-Solà, and Joaquin Garcia-Alfaro. On the difficulty of hiding the balance of Lightning Network channels. In *ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2019.

[26] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash protocol specification, 2016. Electric Coin Company, versions through 2023.

[27] George Kappos, Haaroon Yousaf, Ania Piotrowska, Sanket Kanjalkar, Sergi Delgado-Segura, Andrew Miller, and Sarah Meiklejohn. An empirical analysis of privacy in the Lightning Network. In *Financial Cryptography and Data Security (FC)*, 2021.

[28] Burak Keceli. Ark: An alternative privacy-preserving second layer solution, 2023. bitcoin-dev mailing list, May 2023.

[29] Lightning protocol contributors. BOLT 4: Onion routing protocol, . https://github.com/lightning/bolts/blob/master/04-onion-routing.md, accessed September 30, 2026.

[30] Lightning protocol contributors. BOLT 7: P2P node and channel discovery, . https://github.com/lightning/bolts/blob/master/07-routing-gossip.md, accessed September 30, 2026.

[31] Lightning Service Provider Specification group. LSPS2: JIT channel negotiation (bLIP-52), 2024. https://github.com/lightning/blips.

[32] Robin Linus. BitVM: Compute anything on Bitcoin, 2023. Whitepaper, https://bitvm.org.

[33] Robin Linus, Lukas Aumayr, Alexei Zamyatin, Andrea Pelosi, Zeta Avarikioti, and Matteo Maffei. BitVM2: Bridging Bitcoin to second layers, 2024. Whitepaper, https://bitvm.org.

[34] Robin Linus, Ioannis Alexopoulos, Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, and David Tse. BitVM3: Efficient Bitcoin bridges via garbled circuits, 2026. IACR ePrint 2026/933, revised June 8, 2026.

[35] Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In *CRYPTO*, 2021.

[36] Jonas Nick, Tim Ruffing, and Elliott Jin. BIP 327: MuSig2 for BIP340-compatible multi-signatures, 2022. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki.

[37] Utz Nisslmueller, Klaus-Tycho Foerster, Stefan Schmid, and Christian Decker. Toward active and passive confidentiality attacks on cryptocurrency off-chain networks. *arXiv preprint arXiv:2003.00003*, 2020.

[38] OpenAI. Paired states and Hamiltonian cycles in cubic bipartite planar graphs, 2026. Preprint dated September 24, 2026, in the `openai/math` collection, unrefereed. https://github.com/openai/math/blob/main/preprints/Paired-states-and-Hamiltonian-cycles-in-cubic-bipartite-planar-graphs-September-24-2026/paper.pdf. Lean formalization, not compiled by us: statement in `lean/ComparatorChallenges/BarnetteHamiltonian.lean`, solution in the 31 modules of `lean/OAI/Combinatorics/Hamiltonian/`. Accessed October 7, 2026.

[39] Olaoluwa Osuntokun. LayerTree: A shared Bitcoin UTXO with an O(1) unilateral exit, 2026. Draft dated October 15, 2026, commit ba8422e. https://layertree.tech/paper/main.pdf, accessed October 16, 2026.

[40] Rene Pickhardt and Mariusz Nowostawski. Imbalance measure and proactive channel rebalancing algorithm for the Lightning Network. In *IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2020.

[41] Rene Pickhardt and Stefan Richter. Optimally reliable & cheap payment flows on the Lightning Network. *arXiv preprint arXiv:2107.05322*, 2021.

[42] Joseph Poon and Thaddeus Dryja. The Bitcoin Lightning Network: Scalable off-chain instant payments, 2016. Whitepaper, draft 0.9.0.7.

[43] Matteo Romiti, Friedhelm Victor, Pedro Moreno-Sanchez, Peter Sebastian Nordholt, Bernhard Haslhofer, and Matteo Maffei. Cross-layer deanonymization methods in the Lightning protocol. In *Financial Cryptography and Data Security (FC)*, 2021.

[44] Steven Roose. Post on X quoting the LayerTree announcement, 2026. @stevenroose3, October 16, 2026, quoting @roasbeef's announcement of the same day: that if Bitcoin gets all the opcodes LayerTree requires, Second would build it for production. Read from the timeline; no permalink recorded.

[45] Steven Roose and Erik De Smedt. Evolving the Ark protocol using CTV and CSFS, 2025. Delving Bitcoin, April 15, 2025. https://delvingbitcoin.org/t/evolving-the-ark-protocol-using-ctv-and-csfs/1602.

[46] Jeremy Rubin. BIP-119: CHECKTEMPLATEVERIFY, 2020. https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki.

[47] Gregory Sanders, Antoine Poinsot, and Steven Roose. BIP 446: OP_TEMPLATEHASH, 2026. Draft, assigned February 6, 2026. https://github.com/bitcoin/bips/blob/master/bip-0446.md, accessed October 5, 2026.

[48] Gregory Sanders, Antoine Poinsot, and Steven Roose. BIP 448: Taproot-native (re)bindable transactions, 2026. Draft, assigned March 11, 2026. https://github.com/bitcoin/bips/blob/master/bip-0448.md, accessed October 5, 2026.

[49] Thomas C. Schelling. *The Strategy of Conflict*. Harvard University Press, 1960.

[50] Second. Lightning liquidity: Ark servers and LSPs compared, 2025. https://second.tech/blog/ark-liquidity-research-01, May 1, 2025.

[51] Second. Forfeits & connectors, 2026. Documentation, https://second.tech/docs/learn/forfeits, accessed September 30, 2026.

[52] Second. Ark VTXOs, 2026. Documentation, https://second.tech/docs/learn/vtxo, accessed September 30, 2026.

[53] Second (ark-bitcoin). bark: Ark server (captaind), watchman and client, 2026. https://gitlab.com/ark-bitcoin/bark, master at commit 13ddd156d, October 22, 2026, read October 23, 2026: `server/src/vtxopool.rs` (issuance, defaults, pool inserts), `server/src/ln/mod.rs` (receives funded from the pool), `server/src/watchman/policy.rs` (sweep after expiry), `server/src/round/mod.rs` (no HTLC policies in round outputs).

[54] Gustavus J. Simmons. The prisoners' problem and the subliminal channel. In *Advances in Cryptology: Proceedings of CRYPTO '83*, 1984.

[55] George J. Stigler. A theory of oligopoly. *Journal of Political Economy*, 72(1), 1964.

[56] Sergei Tikhomirov, Rene Pickhardt, Alex Biryukov, and Mariusz Nowostawski. Probing channel balances in the Lightning Network. *arXiv preprint arXiv:2004.00333*, 2020.

[57] Friedhelm Victor and Andrea Marie Weintraud. Detecting and quantifying wash trading on decentralized cryptocurrency exchanges. In *The Web Conference (WWW)*, 2021.

A Notation

Symbol	Meaning
$\rho = (\mathcal{A}, v, F, X, \Lambda, \Gamma, \tau)$	settlement object: authority, value, finality clock, exit, liveness, counterparty set, expiry
$\ell^U, \ell^C, \ell^V, \ell^E$	holdings by class
σ	settlement event (from-class, to-class, amount, clock condition)
O_A, I_A	cumulative outward, inward crossings of ∂A
S_A, S'_A, K_A, K'_A	outbound and inbound stock on ∂A; stock added by capacity events
$\hat{f}_a, \hat{f}_S$	forecast inbound toward agent a; forecast peak aggregate net receipt at server S
R_S, W_S, B_S	server liquidity stock, fronted-and-unrecovered value, total capital
$\bar{L}$	mean residual lock on fronted value at forfeit
$\mathcal{D}_{\text{obj}}(D, H)$	liquidity duration: locked capital-time per unit delivered
$\mathcal{R}_{\text{cap}}, \mathcal{R}_{\text{svc}}$	unique-exposure and service relations
$\text{nf}, \text{cm}, \text{root}_n, k_\rho$	nullifier keyed by the object's authority key, claimant commitment, settled-set root
$E(B, \kappa, \Delta)$	escrow with bond B for connector κ and challenge period Δ
$\mathcal{M}_i, A_i, h$	member set of hyperedge i, its agents, and its hub
$\hat{f}_{A_i}$	forecast peak of the total the agents of hyperedge i hold
R_a	capacity moved toward agent a by reallocation inside a factory, off chain
$\mathcal{D}_k$	liquidity duration of hyperedges of k agents and a hub; $\mathcal{D}_1 = \mathcal{D}_C$
T, P_a, G	the LayerTree leaf; the operator's pre-funded credit leaf for member a; graft value it fronts
T_u, T_ℓ, W	top-up period, lease period and settle window under T
κ, β, $\Delta(q)$	spends of one output per block; adversarial spends ahead; blocks to confirm with q queued
b_T	on-chain bytes per member per block under T
$O_{\text{chain}}, O_{\text{gossip}}, O_{\text{peer}}, O_{\text{server}}$	observer rows

B Reference model

The model accompanying this paper [18] is TypeScript with no dependencies, under `model/`, run with `node -experimental-strip-types -test "model/*.test.ts"`. It implements the extended ledger with clocked settlement events (Proposition 2); the server liquidity ledger under both recovery readings, the refresh cycle, the aggregate bound, and the server-tier term, with the channel model under oracle and trailing forecasts and the grid of Table 4 (Proposition 3); the multiparty channel on the same demand, checked to coincide with the channel model at $k = 1$, and the tables of Section 5 the curves of Figure 6, the sample paths of Figure 2, and the pooling-scale Monte Carlo of Table 1 (Proposition 4, Remark 1); the LayerTree draft's cost tables on its own transaction model, reproducing its Table 5 (exits by variant, in the model's vB and not the serialised) and Table 10 (lifetime bytes) exactly from its witness weights and skeleton, while the 122 and 151 vB recurring figures are carried from its text, with the contention bound of Equation (4), the fee floors, the terms of Equation (3) and the break-even receipt rate of Section 4; the epoched registry with authority-keyed nullifiers and attested successors (Propositions 5, 6); the escrow

state machine with the published BitVM3 sizes as inputs (Proposition 7); and three warden statistics on a coalition fixture, with the two that fail asserted to fail. It also holds the witness of Appendix C: three cubic bipartite polyhedral graphs, a Hamiltonian cycle avoiding each edge in turn found by exhaustive search, and the unit circulation round it. The proof system is not implemented; the registry verifies a simulated attestation with the interface a verifier would expose. Twenty-five tests pass at the cited commit; they witness the propositions on finite cases and prove nothing about distributed execution. Three reference-model corrections of October 22, 2026, found by the independent Lean 4 project that formalises the accounting core [16], each carry a regression test that fails on the prior code: route application was not atomic on routes that reuse a channel; the Ark server's outstanding capital was charged across an interval before the sweeps inside it; and the mock settlement set's encoding was ambiguous. The tables of Sections 4 and 5 were regenerated after the corrections and did not change at the printed precision.

C A remark on designed topologies

Nothing in this appendix is about the Lightning Network as it is. The public channel graph is not cubic, not bipartite, and not planar, and the theorem recalled below says nothing about it. The remark concerns an overlay that someone designs, such as the channel set an Ark server places (Section 4) or a factory whose members a warden admits (Section 5), where the shape of the graph is a choice. It records one property such a shape can be given, what the property amounts to in the terms of this paper, and what it costs. No proposition in the main text depends on it.

The result and its standing. A preprint by OpenAI [38] claims Barnette's conjecture: every finite simple cubic bipartite planar 3-vertex-connected graph has a Hamiltonian cycle (its Theorem 1.1). Its Corollary 7.1 adds that for every edge there is a Hamiltonian cycle avoiding that edge. The manuscript is an unrefereed, machine-generated preprint. We checked the statements of Theorem 1.1 and Corollary 7.1 against the manuscript and read its outline; we have not refereed the proof. A Lean formalization is published beside it: thirty-one modules in which a text scan finds no `sorry`, `admit`, or added axiom, reached from a challenge file that states the theorem, leaves its own proof as `sorry`, and names those modules as the solution. We compiled neither. No outside review is known to us. The paper says of its argument that it "does not assert a polynomial-time construction of the cycle." Everything below is therefore conditional on a claim whose checking is incomplete, and is written so that nothing else in the paper leans on it.

What it would give a designed overlay. Let the overlay be a channel graph in which every participant holds exactly three channels, the participants fall into two classes with every channel between the classes, the graph is planar, and no two participants' removal disconnects it. If Theorem 1.1 holds, there is a closed route that visits every participant once. In the notation of Appendix A, a payment of x from each participant to its successor along the route is a settled event on each channel of the route and on no other. Every participant's net position is unchanged, since each pays x and receives x; on each route channel x moves from the upstream to the downstream side; and each participant's spendable balance is thereby rotated from its successor channel to its predecessor channel. If Corollary 7.1 holds, such a route exists that avoids any one channel named in advance, so after any single channel closes or is left out of the design some route remains, though

not the same one. That is the whole of what the theorem supplies: the existence of one spanning route for a uniform circulation. It does not supply a route through a channel the designer names, and it does not supply the several routes that an arbitrary pattern of imbalances would need, since one circulation shifts every route channel by the same amount in the same rotational sense. The paper's Corollary 7.2, which places every three-edge path on some Hamiltonian cycle for a class that includes the planar case, would give routes through a named channel; it rests on an equivalence the paper takes from elsewhere, and we do not use it.

Limits. Three. Existence is not construction: the argument asserts no polynomial-time construction, so a designer who wants the route has to search for it, and on an overlay of any size the theorem promises only that the search will end. The shape is expensive in the terms of Section 3: every participant holds three two-party channels, each direction-partitioned, so the forecast of Section 3 is made three times per participant where the star of Section 4 makes it once and the hyperedge of Section 5 once per group; a cubic bipartite graph has classes of equal size, so if one class is agents and the other supplies their liquidity, the design needs as many suppliers as agents; and nothing in it pools, so Remark 1 offers it nothing. The route is a property the public graph cannot be given, bought at roughly three times the per-agent channel's capital per participant. And a route that exists as a cycle of the graph need not be feasible at a given amount. Each channel on it has to hold at least x on its upstream side when the payment is made, which is the condition of Lemma 1 on each edge of the ring, and a circulation feasible in one rotational sense may not be in the other. Dugan [17] makes the general point against the earlier manuscript's cut reasoning: feasibility depends on every prefix of the arrival sequence and on which edge the liquidity sits, so that two histories with the same totals and the same final imbalance can differ in what they serve, and a bound on a cut is not a certificate of delivery. The theorem speaks to the graph and not to the balances on it, and the balances are what Sections 3 to 5 are about.

Witness. The reference model checks the statement on three small graphs of the class, the cube, the hexagonal prism, and the truncated octahedron: for each edge in turn it finds a Hamiltonian cycle avoiding that edge by exhaustive search, and checks that a unit circulation round the cycle nets to zero at every participant, moves one unit across each cycle channel, and moves nothing elsewhere (Appendix B). It witnesses the statement on finite cases and proves nothing further.